



南京航空航天大学
NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS

人工智能内容生成 (AIGC) 技术与应用



第六章

自回归生成模型

典型自回归网络结构-RNN

RNN的具体表达式

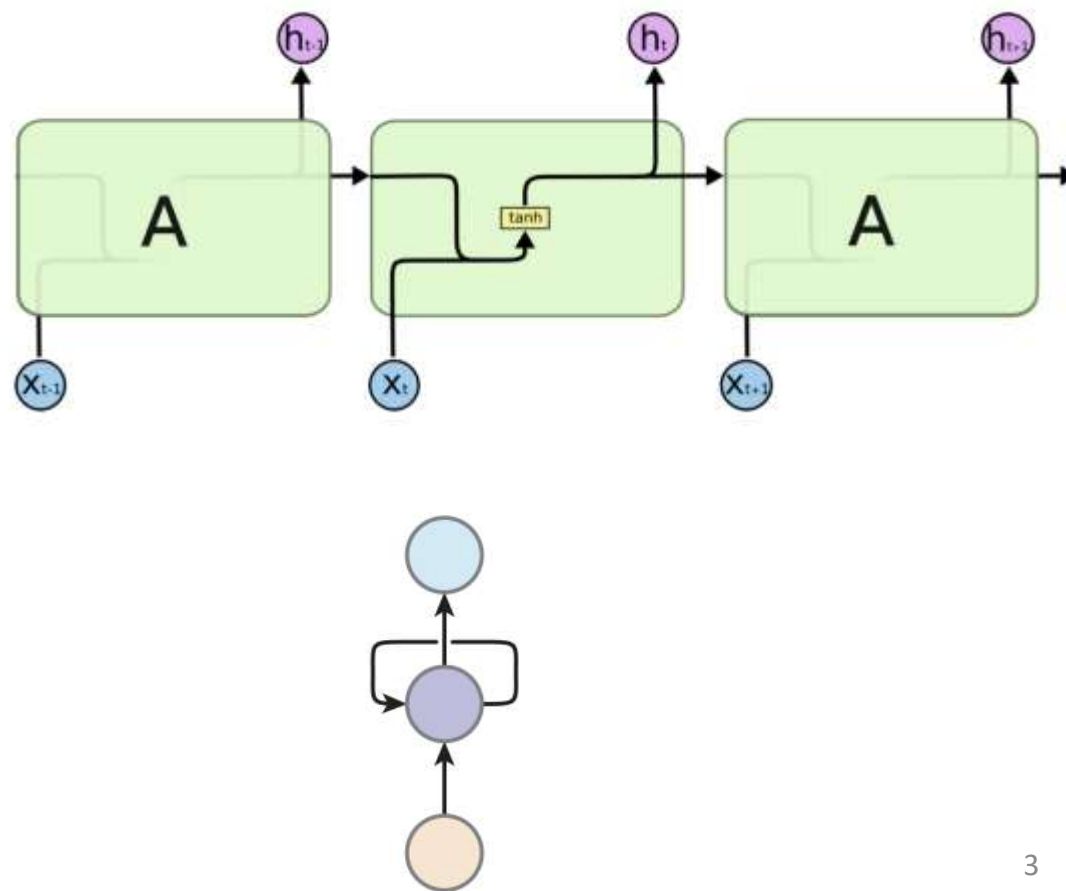
- 隐藏状态更新公式

$$h_t = f(W_{ih} \cdot x_t + W_{hh}h_{t-1} + b_h)$$

- 输出层公式:

$$y_t = g(w_{hy} \cdot h_t + b_y)$$

其中, f 是隐藏层的激活函数, g 是输出层的激活函数。



典型自回归网络结构-RNN

以Recipes数据集为例，构建LSTM（RNN）模型

20000个食谱文本数据集

```
with open('/app/data/epirecipes/full_format_recipes.json') as json_data:
    recipe_data = json.load(json_data)

filtered_data = [
    'Recipe for ' + x['title'] + ' | ' + ' '.join(x['directions'])
    for x in recipe_data
    if 'title' in x
    and x['title'] is not None
    and 'directions' in x
    and x['directions'] is not None
]
```

数据加载代码

Recipe for Ham Persillade with Mustard Potato Salad and Mashed Peas | Chop enough parsley leaves to measure 1 tablespoon; reserve. Chop remaining leaves and stems and simmer with broth and garlic in a small saucepan, covered, 5 minutes. Meanwhile, sprinkle gelatin over water in a medium bowl and let soften 1 minute. Strain broth through a fine-mesh sieve into bowl with gelatin and stir to dissolve. Season with salt and pepper. Set bowl in an ice bath and cool to room temperature, stirring. Toss ham with reserved parsley and divide among jars. Pour gelatin on top and chill until set, at least 1 hour. Whisk together mayonnaise, mustard, vinegar, 1/4 teaspoon salt, and 1/4 teaspoon pepper in a large bowl. Stir in celery, cornichons, and potatoes. Pulse peas with marjoram, oil, 1/2 teaspoon pepper, and 1/4 teaspoon salt in a food processor to a coarse mash. Layer peas, then potato salad, over ham.

输出样例

文本数据 VS 图像数据

文本 (Text)

图像 (Image)

数据形态

离散的单元 (字符/单词)

连续的色彩光谱中的点

维度与顺序

时间维度 (语序重要)

空间维度 (像素并行)

敏感度

对微小变化高度敏感

对个别单元变化不敏感

结构规则

遵循语法和语义规则

无既定 “语法规则”

典型自回归网络结构-RNN

词元化 (Tokenization)

将文本分割成独立单元（例如单词）的过程

- **大小写归一**：消除句首大写差异（如

The = the)

- **词干提取**：将单词还原为词根（如

browsing → brows)

作用：合并不同时态，简化特征

- **标点符号**

- 二选一：作为词元保留 或 完全移除

```
def pad_punctuation(s):  
    s = re.sub(f"([string.punctuation])", r' \1 ', s)  
    s = re.sub(' +', ' ', s)  
    return s  
  
text_data = [pad_punctuation(x) for x in filtered_data] ❶  
  
text_ds = tf.data.Dataset.from_tensor_slices(text_data).batch(32).shuffle(1000) ❷  
  
vectorize_layer = layers.TextVectorization( ❸  
    standardize = 'lower',  
    max_tokens = 10000,  
    output_mode = "int",  
    output_sequence_length = 200 + 1,  
)  
  
vectorize_layer.adapt(text_ds) ❹  
vocab = vectorize_layer.get_vocabulary() ❺
```

- **词汇表优化**

- 使用高频词作为词汇表
- 使用“未知词标记”（如 <UNK>）
统一替换稀疏词。

典型自回归网络结构-RNN

词元化 (Tokenization)

将文本分割成独立单元（例如单词）的过程

Recipe for Ham Persillade with Mustard Potato Salad and Mashed Peas | Chop enough parsley leaves to measure 1 tablespoon; reserve. Chop remaining leaves and stems and simmer with broth and garlic in a small saucepan, covered, 5 minutes. Meanwhile, sprinkle gelatin over water in a medium bowl and let soften 1 minute. Strain broth through a fine-mesh sieve into bowl with gelatin and stir to dissolve. Season with salt and pepper. Set bowl in an ice bath and cool to room temperature, stirring. Toss ham with reserved parsley and divide among jars. Pour gelatin on top and chill until set, at least 1 hour. Whisk together mayonnaise, mustard, vinegar, 1/4 teaspoon salt, and 1/4 teaspoon pepper in a large bowl. Stir in celery, cornichons, and potatoes. Pulse peas with marjoram, oil, 1/2 teaspoon pepper, and 1/4 teaspoon salt in a food processor to a coarse mash. Layer peas, then potato salad, over ham.

```
[ 26 16 557 1 8 298 335 189 4 1054 494 27 332 228
 235 262 5 594 11 133 22 311 2 332 45 262 4 671
 4 70 8 171 4 81 6 9 65 80 3 121 3 59
 12 2 299 3 88 650 20 39 6 9 29 21 4 67
 529 11 164 2 320 171 102 9 374 13 643 306 25 21
 8 650 4 42 5 931 2 63 8 24 4 33 2 114
 21 6 178 181 1245 4 60 5 140 112 3 48 2 117
 557 8 285 235 4 200 292 980 2 107 650 28 72 4
 108 10 114 3 57 204 11 172 2 73 110 482 3 298
 3 190 3 11 23 32 142 24 3 4 11 23 32 142
 33 6 9 30 21 2 42 6 353 3 3224 3 4 150
 2 437 494 8 1281 3 37 3 11 23 15 142 33 3
 4 11 23 32 142 24 6 9 291 188 5 9 412 572
 2 230 494 3 46 335 189 3 20 557 2 0 0 0
 0 0 0 0 0]
```

0:[STOPTOKEN]

1:[Unknown Token]



创建数据集

- 输入词元序列向后移动一个标记变成输出

```
def prepare_inputs(text):  
    text = tf.expand_dims(text, -1)  
    tokenized_sentences = vectorize_layer(text)  
    x = tokenized_sentences[:, :-1]  
    y = tokenized_sentences[:, 1:]  
    return x, y  
  
train_ds = text_ds.map(prepare_inputs) ❶
```


典型自回归网络结构-RNN

词元嵌入层 (Embedding Layer)

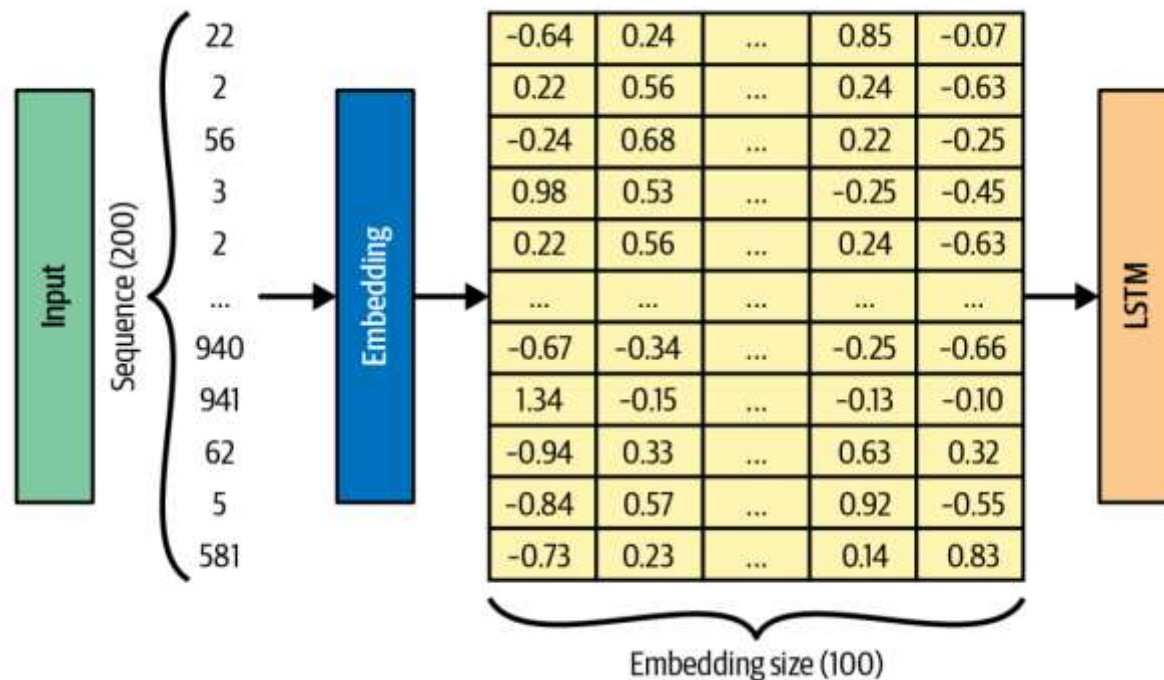
- 一个可学习的查找表 (Lookup Table), 将整数词元 (token) 映射为固定长度的向量权重。

Vocabulary size (10,000)

Token	Embedding				
0	-0.13	0.45	...	0.13	-0.04
1	0.22	0.56	...	0.24	-0.63
...	...	...	...	...	...
9998	0.16	-0.70	...	-0.35	1.02
9999	-0.98	-0.45	...	-0.15	-0.52

Embedding size (100)

参数总量=词汇表大小 (Vocabulary Size) ×
嵌入维度 (Embedding Dimension)





LSTM 训练

```
inputs = layers.Input(shape=(None,), dtype="int32") ❶
x = layers.Embedding(10000, 100)(inputs) ❷
x = layers.LSTM(128, return_sequences=True)(x) ❸
outputs = layers.Dense(10000, activation = 'softmax')(x) ❹
lstm = models.Model(inputs, outputs) ❺

loss_fn = losses.SparseCategoricalCrossentropy()
lstm.compile("adam", loss_fn) ❻
lstm.fit(train_ds, epochs=25) ❼
```

- **输入层**: 使用 None 作为占位符, 支持灵活可变的序列长度。
- **嵌入层**: 设定词汇表大小为 10,000, 嵌入向量维度为 100。
- **LSTM 层**: 隐藏向量维度设为 128, 并输出完整的时间步序列而非仅最后一步。
- **全连接层**: 将每个时间步的隐藏状态转换为下一个 Token 的概率向量。
- **模型目标**: 针对序列中的每个 Token, 预测其下一个 Token。
- **损失函数**: 选用 SparseCategoricalCrossentropy, 直接处理整数标签而非 One-Hot 向量。

典型自回归网络结构-RNN

LSTM 训练

Epoch 1/25

628/629 [=====>.] - ETA: 0s - loss: 4.4536

generated text:

recipe for mold salad are high 8 pickled to fold cook the dish into and warm in baking reduced but halves beans and cut

629/629 [=====] - 29s 43ms/step - loss: 4.4527

Epoch 2/25

628/629 [=====>.] - ETA: 0s - loss: 3.2339

generated text:

recipe for racks - up-don with herb fizz | serve checking thighs onto sanding butter and baking surface in a lvy heavy large saucepan over blender ; stand overnight . [UNK] over moderate heat until very blended , garlic about 8 minutes . cook airtight until cooked are soft seeds , about 1 45 minutes . sugar , until s is brown , to sliced , parmesan , until browned and add extract . wooden crumb to outside of out sheets . flatten and preat return to the paste . add in pecans oval and let transfer day .

629/629 [=====] - 30s 48ms/step - loss: 3.2336

Epoch 3/25

629/629 [=====] - ETA: 0s - loss: 2.6229

generated text:

recipe for grilled chicken | preheat oven to 400°f . cook in large 8 - caramel grinder or until desired are f , about 6 minutes

629/629 [=====] - 27s 42ms/step - loss: 2.6229

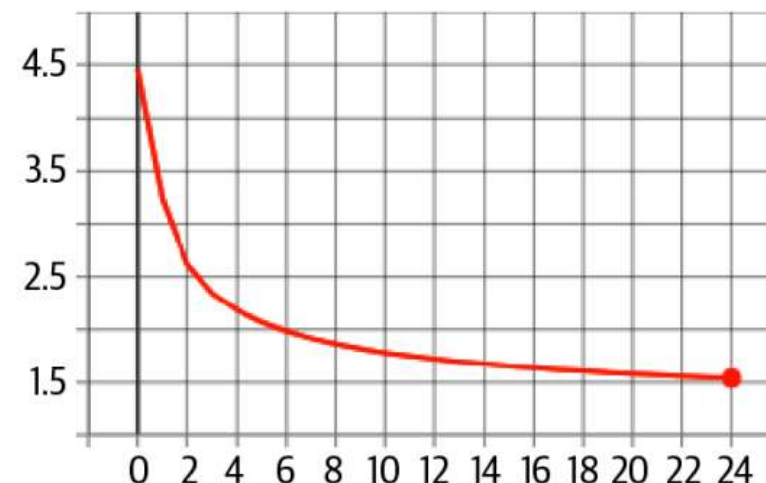
Epoch 4/25

629/629 [=====] - ETA: 0s - loss: 2.3426

generated text:

recipe for pizza salad with sweet red pepper and star fruit | combine all ingredients except lowest ingredients in a large skillet . working with batches and deglaze , cook until just cooked through , about 1 minute . meanwhile , boil potatoes and paprika in a little oil over medium - high heat , stirring it just until crisp , about 3 minutes . stir in bell pepper , onion and cooked paste and jalapeño until clams well after most - reggiano , abo ut 5 minutes . transfer warm 2 tablespoons flesh of eggplants to medium bowl . serve .

epoch_loss
tag: epoch_loss



文本生成

➤ 主要流程:

- **预测下一个词:** 将现有的单词序列输入网络, 让模型预测紧接其后的下一个单词。
- **循环迭代:** 将新预测出的单词追加到序列末尾, 作为新的输入重复上述过程以生成后续内容

```
class TextGenerator(callbacks.Callback):  
    def __init__(self, index_to_word, top_k=10):  
        self.index_to_word = index_to_word  
        self.word_to_index = {  
            word: index for index, word in enumerate(index_to_word)  
        } ①  
  
    def sample_from(self, probs, temperature): ②  
        probs = probs ** (1 / temperature)  
        probs = probs / np.sum(probs)  
        return np.random.choice(len(probs), p=probs), probs  
  
    def generate(self, start_prompt, max_tokens, temperature):  
        start_tokens = [  
            self.word_to_index.get(x, 1) for x in start_prompt.split()  
        ] ③  
        sample_token = None  
        info = []  
        while len(start_tokens) < max_tokens and sample_token != 0: ④  
            x = np.array([start_tokens])  
            y = self.model.predict(x) ⑤
```

- **建立映射:** 创建反向词汇映射表, 实现从单词到 Token 索引的转换。
- **温度调节:** 定义概率调整函数, 利用温度参数缩放预测分布。
- **处理输入:** 将初始提示词转换为 Token 序列, 作为生成的起始点。
- **设定终止:** 循环生成直到达到最大长度限制或遇到停止符。
- **模型预测:** 模型输出词汇表中每个单词成为序列下一个词的概率分布。
- **采样输出:** 基于温度参数通过采样器从概率中选择下一个单词。
- **迭代更新:** 将新生成的单词追加到当前序列末尾, 作为下一轮预测的输入。

典型自回归网络结构-RNN

文本生成

使用不同的温度系数生成文本

➤ **风格一致性**: 两个样本均模仿原始训练集的食谱风格, 包含标题且语法基本正确。

➤ **温度差异 (Temperature = 1.0)**:

- 生成内容更具冒险性与创新性。
- 准确性相对较低。
- 由于概率分布方差更大, 多次采样能产生更多样化的结果。

➤ **温度差异 (Temperature = 0.2)**:

- 生成内容相对保守。
- 准确性更高, 但变化较少。

酸味土豆?

temperature = 1.0

generated text:

recipe for sour japanese potatoes julienne in a bowl stir together the yeast mixture with the milk and the peanut butter crumbs , the sour cream , and the butter mixture with a fork , gently fold in the prunes gently or until incorporated . lightly stir the oil and yeast until it just holds soft peaks , but not runny , on bottom of a 7 - sided sheet of aluminum foil , top it with a round , and a pinch of each brownies into a goblet , or with the baking dish . serve each with sorbet

temperature = 0.2

generated text:

recipe for grilled chicken with mustard - herb sauce | combine first 6 ingredients in medium bowl . add chicken to pot . add chicken and turn to coat . cover and refrigerate at least 1 hour and up to 1 day . preheat oven to 450°f . place turkey on rack in roasting pan . roast until thermometer inserted into thickest part of thigh registers 175°f , about 1 hour longer . transfer to rack in center of oven and preheat to 450°f . brush chicken with oil . sprinkle with salt and pepper . roast until thermometer inserted into

文本生成

➤ 语法与语境能力

- **词性自适应**：无需人工标注，模型能自动区分词性，生成符合语法的句子。
- **上下文感知**：能根据标题匹配合适的动词，如“烤蔬菜” → “预热”，“冰淇淋” → “搅拌”。

temperature = 1.0	
PROMPT: recipe for roast	
turkey:	22.81%
chicken:	19.41%
beef:	10.24%
pork:	9.96%
leg:	4.06%

PROMPT: recipe for roasted vegetables	
preheat:	69.63%
prepare:	3.68%
heat:	3.45%
put:	2.12%
combine:	1.96%

PROMPT: recipe for chocolate ice cream	
in:	27.31%
combine:	11.21%
stir:	6.66%
whisk:	5.64%
mix:	3.68%

PROMPT: recipe for roasted vegetables chop 1 /	
2:	53.51%
4:	29.83%
3:	13.11%
8:	0.78%
1:	0.56%

temperature = 0.2	
PROMPT: recipe for roast	
turkey:	67.54%
chicken:	30.15%
beef:	1.23%
pork:	1.07%
leg:	0.01%

PROMPT: recipe for roasted vegetables	
preheat:	100.0%
prepare:	0.0%
heat:	0.0%
put:	0.0%
combine:	0.0%

PROMPT: recipe for chocolate ice cream	
in:	98.71%
combine:	1.15%
stir:	0.09%
whisk:	0.04%
mix:	0.0%

PROMPT: recipe for roasted vegetables chop 1 /	
2:	94.81%
4:	5.11%
3:	0.08%
8:	0.0%
1:	0.0%

不同温度系数下，针对不同提示词，概率最高的5个词元

文本生成

➤ 温度参数的影响

- **低温度 ($T=0.2$)**: 概率高度集中于最优选项, 生成内容更稳定、单一, 随机性降低。

➤ 局限与改进

- **语义缺失**: 缺乏深层理解, 常生成不合理搭配 (如 “酸味土豆+冰棍”) 。
- **改进方向**: 需增强词汇组合逻辑理解与长时记忆能力。

temperature = 1.0	
PROMPT: recipe for roast	
turkey:	22.81%
chicken:	19.41%
beef:	10.24%
pork:	9.96%
leg:	4.06%

PROMPT: recipe for roasted vegetables	
preheat:	69.63%
prepare:	3.68%
heat:	3.45%
put:	2.12%
combine:	1.96%

PROMPT: recipe for chocolate ice cream	
in:	27.31%
combine:	11.21%
stir:	6.66%
whisk:	5.64%
mix:	3.68%

PROMPT: recipe for roasted vegetables chop 1 /	
2:	53.51%
4:	29.83%
3:	13.11%
8:	0.78%
1:	0.56%

temperature = 0.2	
PROMPT: recipe for roast	
turkey:	67.54%
chicken:	30.15%
beef:	1.23%
pork:	1.07%
leg:	0.01%

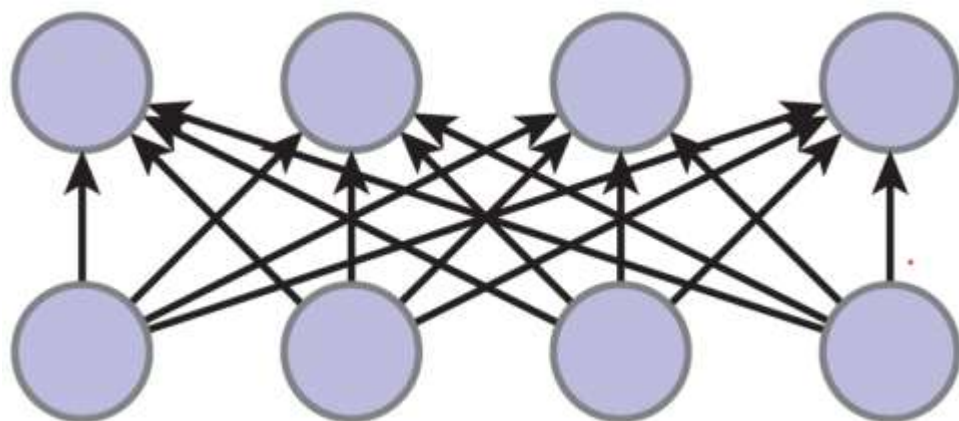
PROMPT: recipe for roasted vegetables	
preheat:	100.0%
prepare:	0.0%
heat:	0.0%
put:	0.0%
combine:	0.0%

PROMPT: recipe for chocolate ice cream	
in:	98.71%
combine:	1.15%
stir:	0.09%
whisk:	0.04%
mix:	0.0%

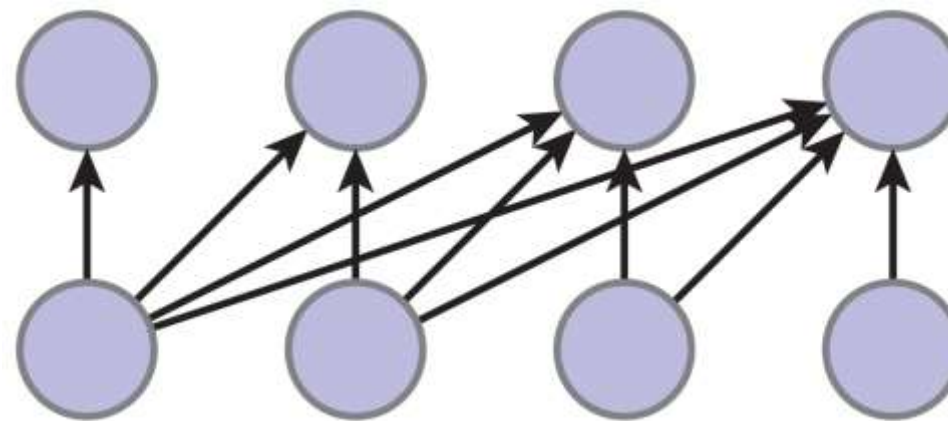
PROMPT: recipe for roasted vegetables chop 1 /	
2:	94.81%
4:	5.11%
3:	0.08%
8:	0.0%
1:	0.0%

不同温度系数下, 针对不同提示词, 概率最高的5个词元

简介：用于自回归的Self Attention



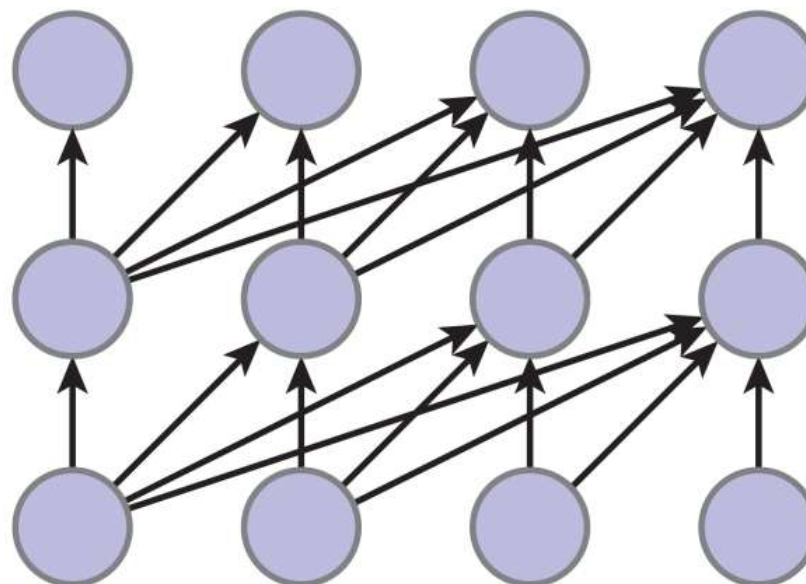
全注意力
(每一步都能看到所有步骤)



因果注意力
(不依赖于“未来”信息)

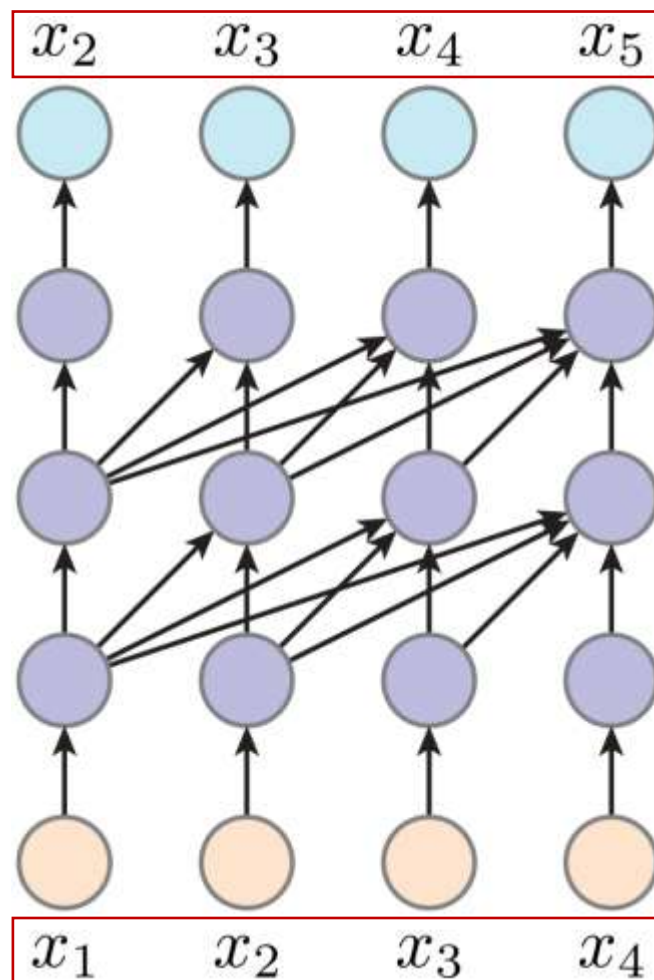
简介：用于自回归的Self Attention

增加深度



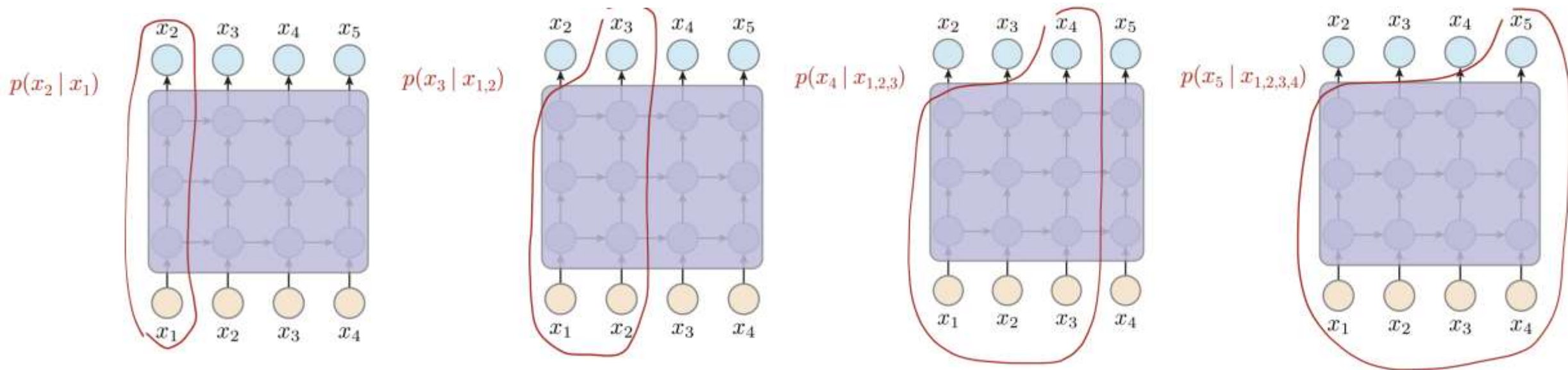
简介：用于自回归的Self Attention

将目标值向后偏移一步



典型自回归网络结构

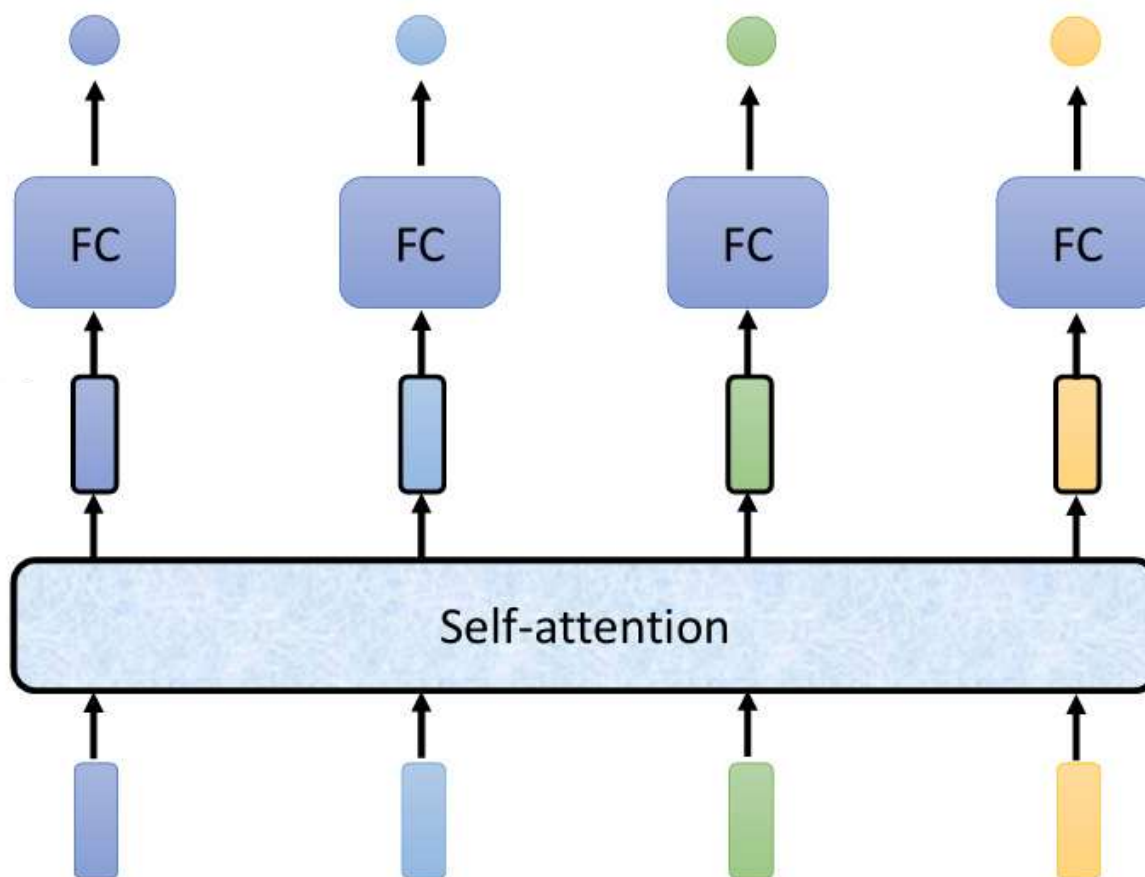
简介：用于自回归的Self Attention



典型自回归网络结构-Attention

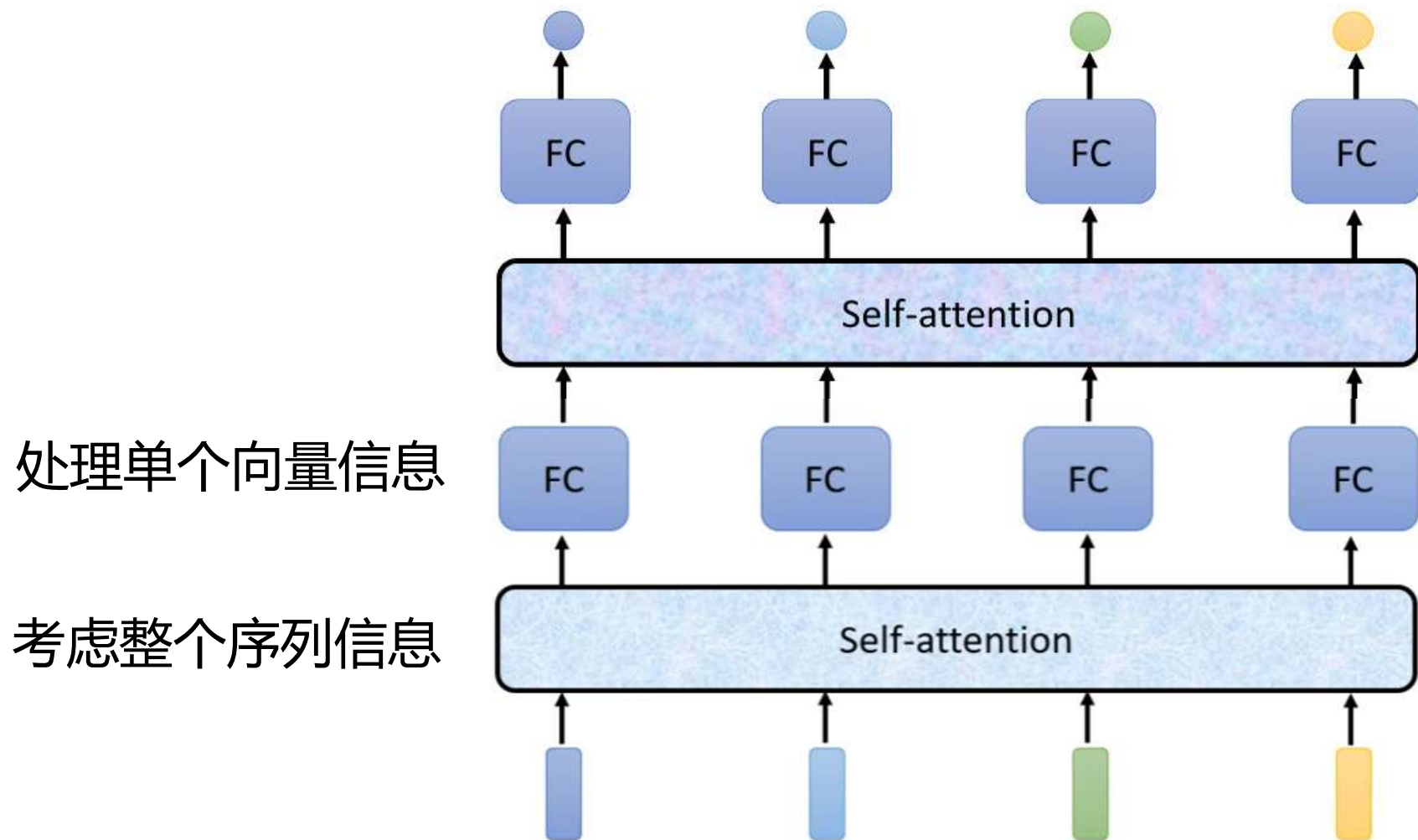
自注意力网络 (Self-Attention)

Self-attention输入N个向量输出N个向量，每个向量受整个序列的上下文影响



典型自回归网络结构-Attention

Self-attention可以叠加很多次，交替使用

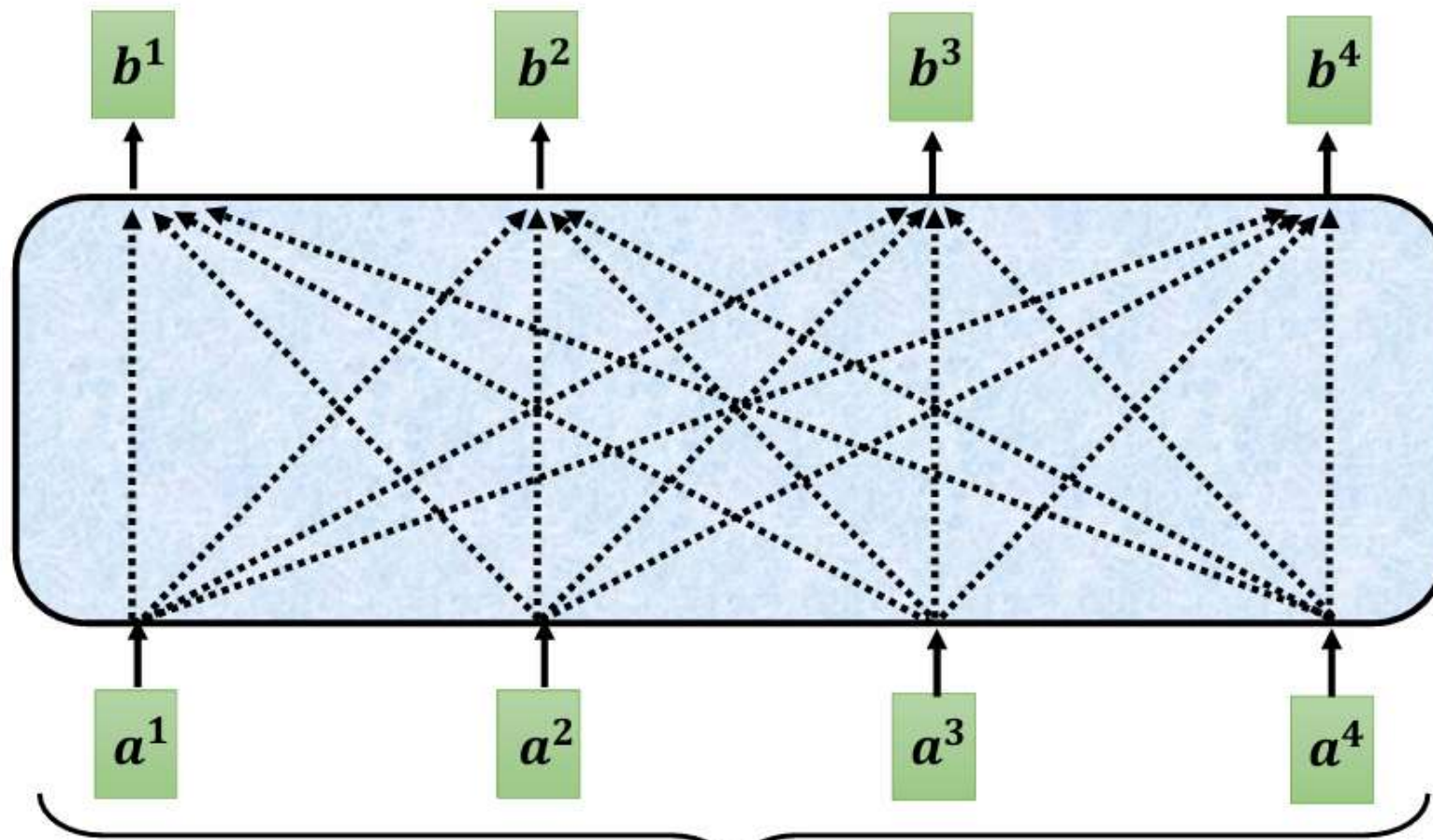


典型自回归网络结构-Attention

Self-attention详细结构是什么样子的？

每个 b^x 考虑
 $\{a^1, \dots, a^4\}$ 整个序列

注意：先不考虑条件
依赖关系。
后续会加入条件依
赖约束

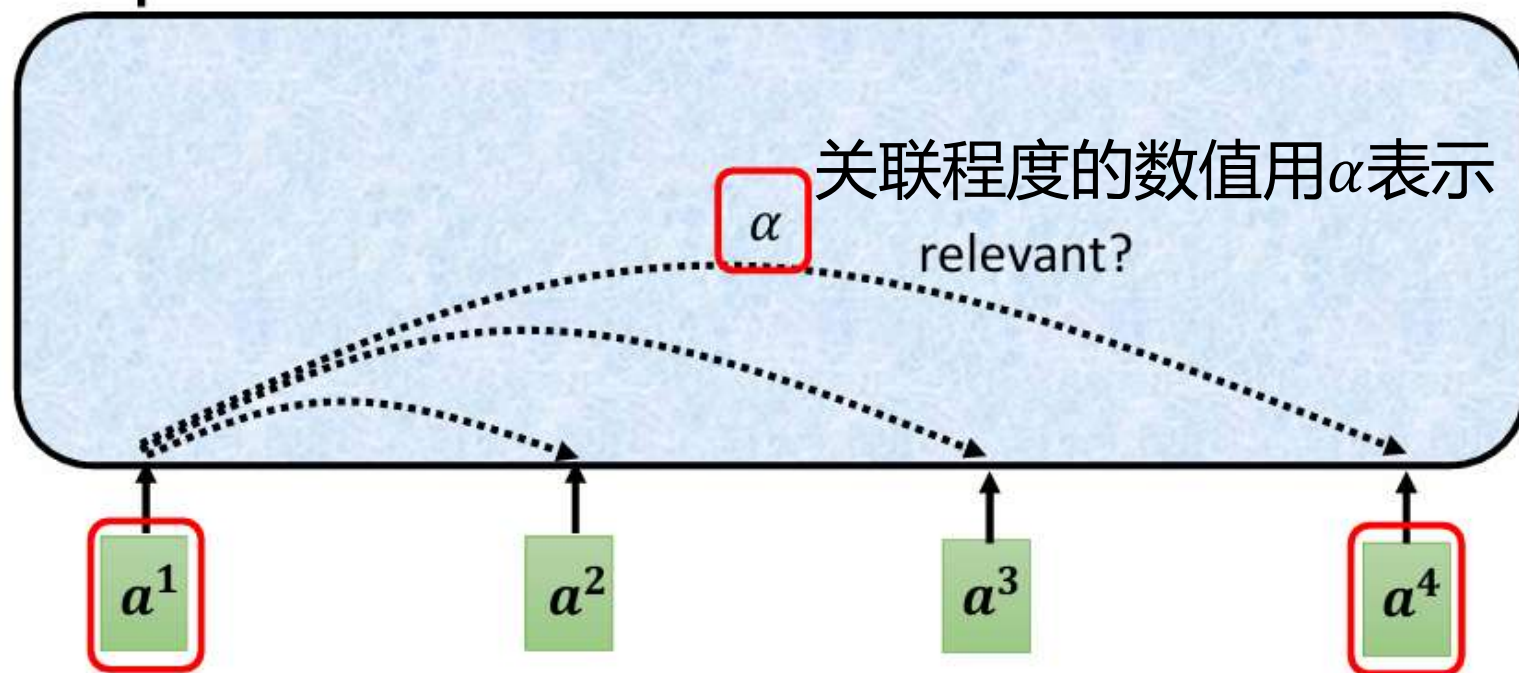


可以输入一个向量/一个隐藏层的输出

典型自回归网络结构-Attention

Self-attention每个输出向量 b^x 是怎么得到的?

步骤一 b^1 以 b^1 为例, 根据 a^1 , 找出序列里和 a^1 相关的其他向量, 代替先前的 “用window把整个输入包起来”



如何刻画每个向量之间的关联程度 α

典型自回归网络结构-Attention

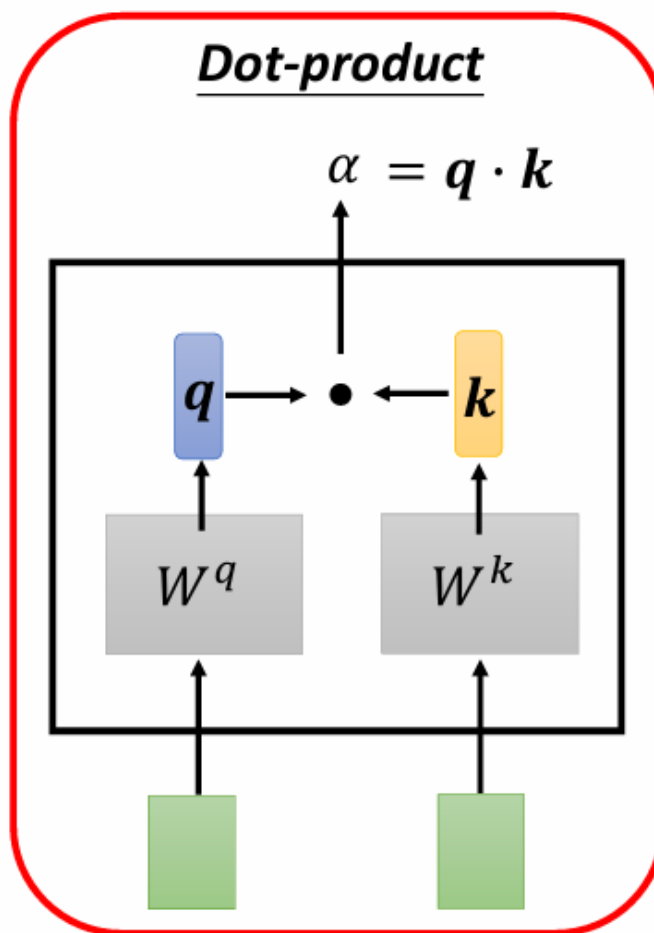
如何刻画每个向量之间的关联程度 α

➤ 方法一

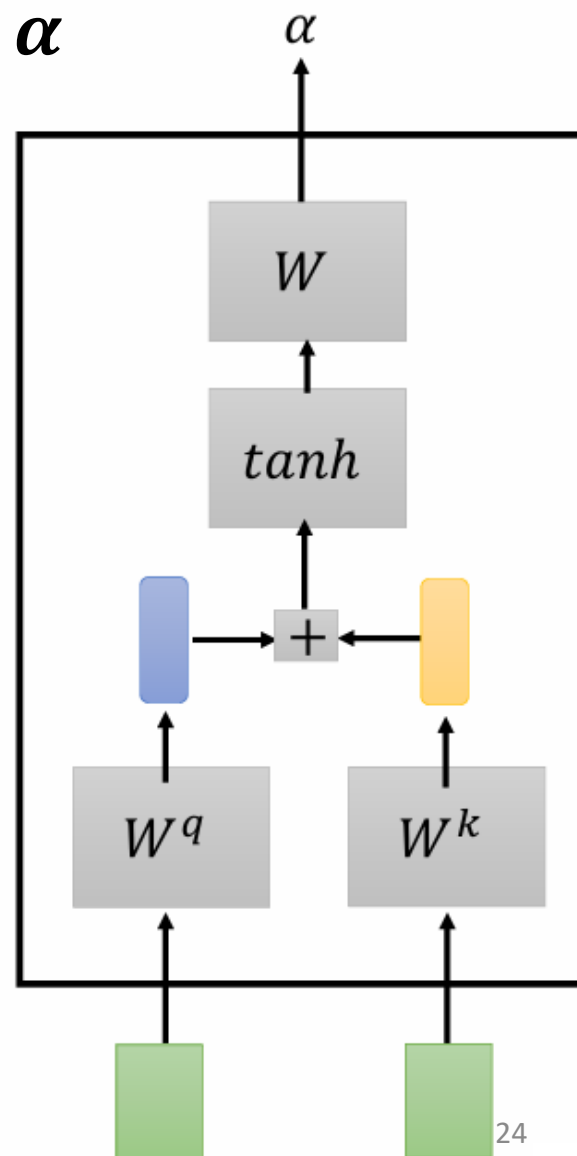
两个比较的输入向量分别乘两个矩阵 W^q , W^k 得到 q , k 向量, 再点积得到 α

➤ 方法二

两个比较的输入向量分别乘两个矩阵 W^q , W^k 得到 q , k 向量, 拼接后过激活函数 ($\tanh$), 再乘 W 矩阵得到 α

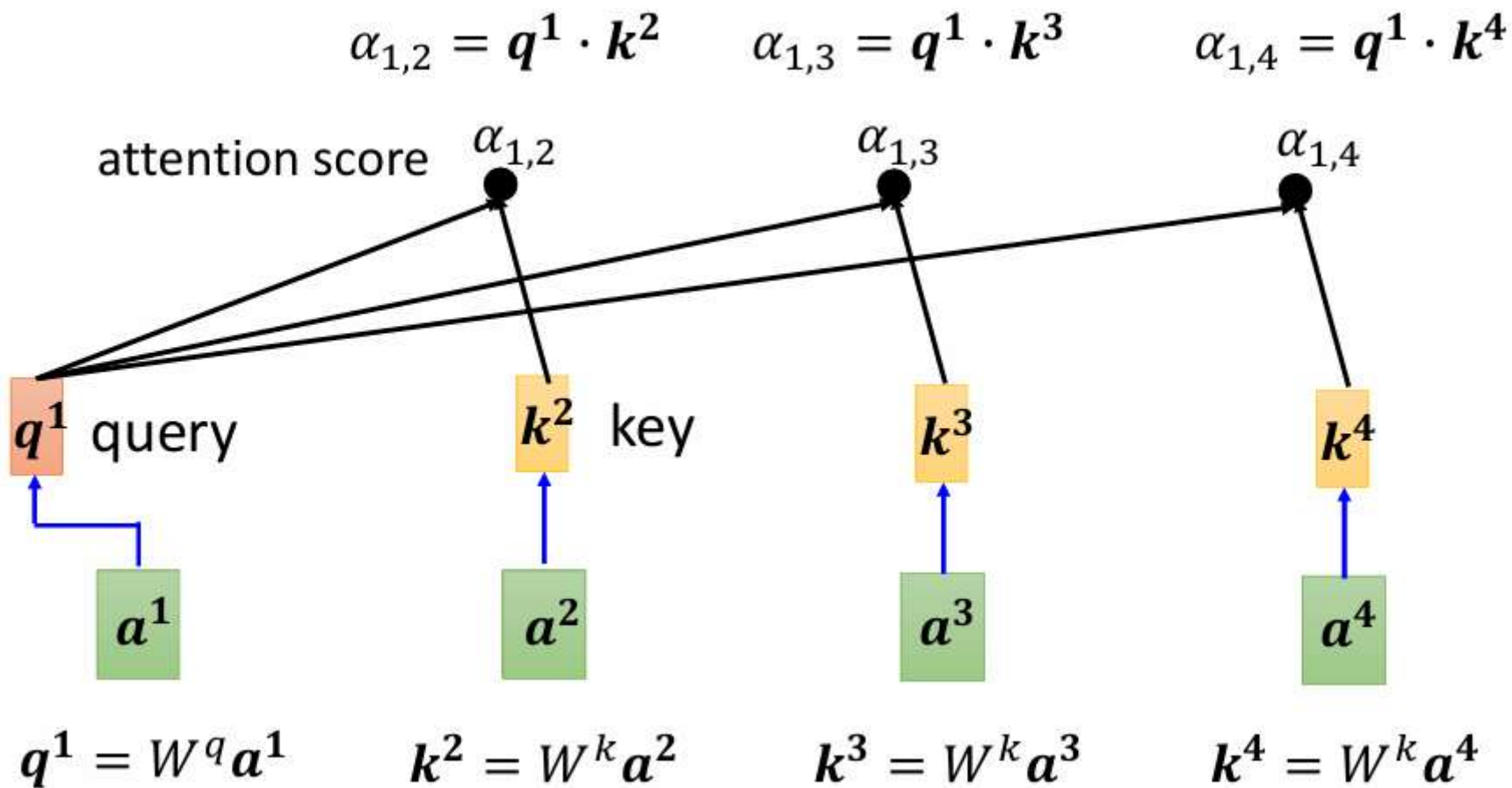
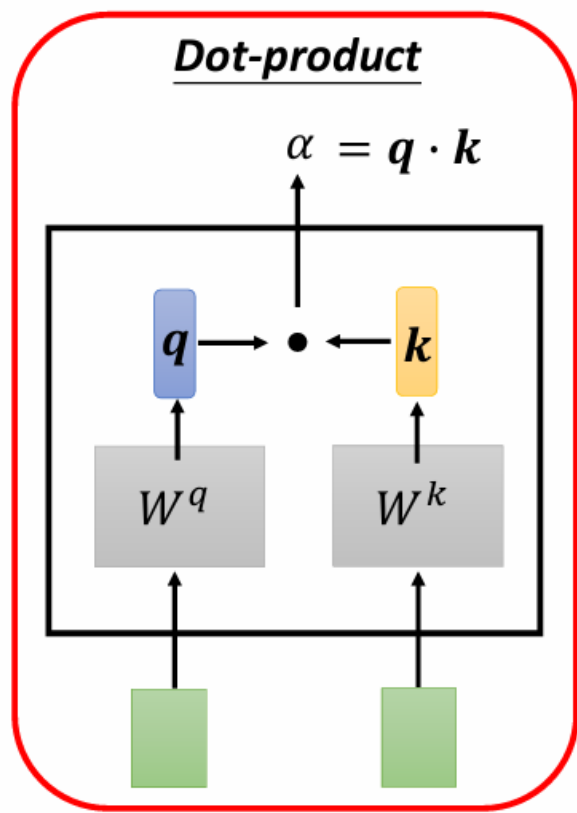


Additive



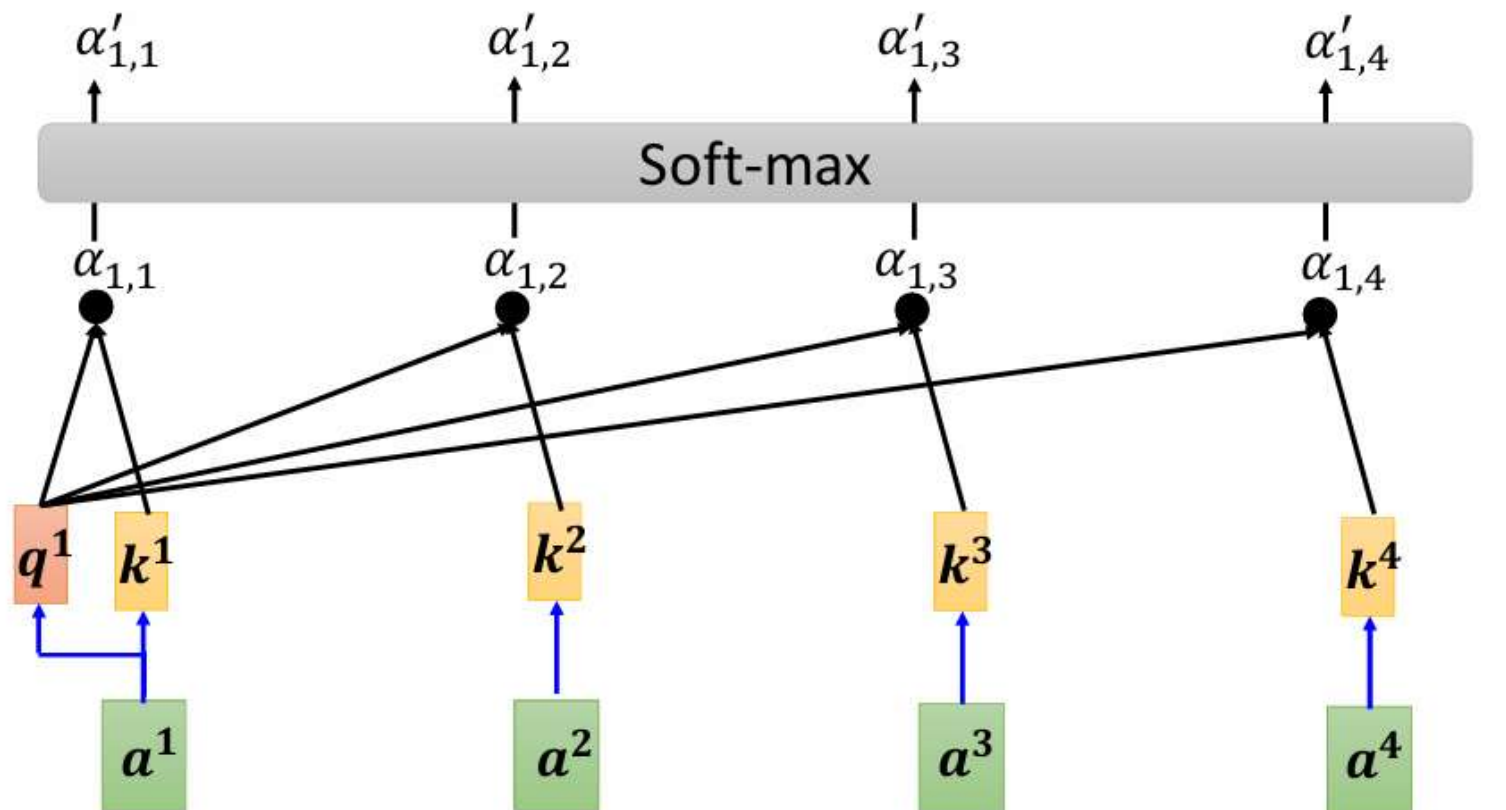
典型自回归网络结构-Attention

如何刻画每个向量之间的关联程度 α



典型自回归网络结构-Attention

实际应用中 $a(\text{query})$ 也会和自己做关联性计算



$$q^1 = W^q a^1$$

$$k^1 = W^k a^1$$

$$k^2 = W^k a^2$$

$$k^3 = W^k a^3$$

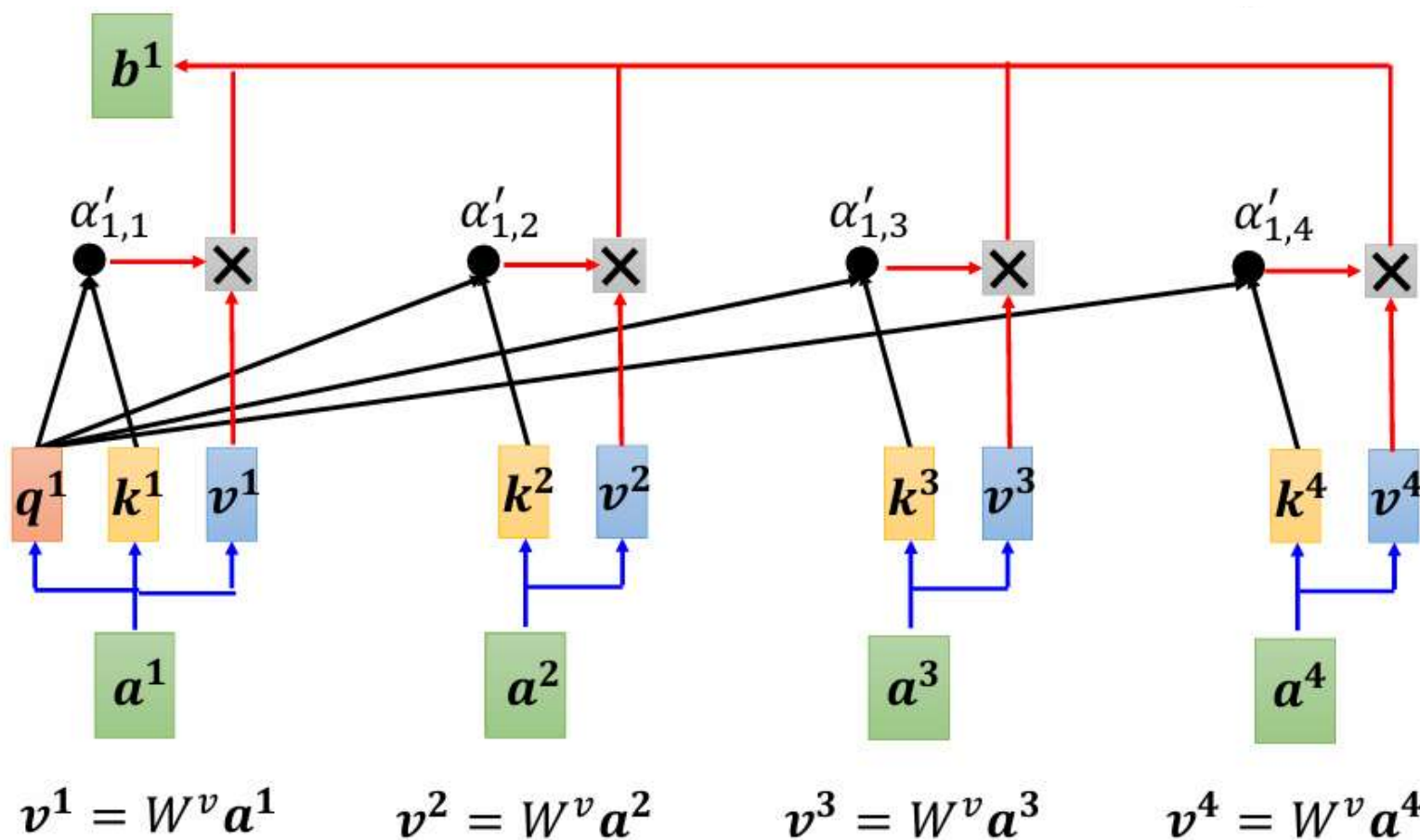
$$k^4 = W^k a^4$$

$$\alpha'_{1,i} = \exp(\alpha_{1,i}) / \sum_j \exp(\alpha_{1,j})$$

query和自己做关联性计算的意义可以形象的理解为建立一个标准, query和本身的关联性应该是最大的

典型自回归网络结构-Attention

根据相关性程度提取关键信息



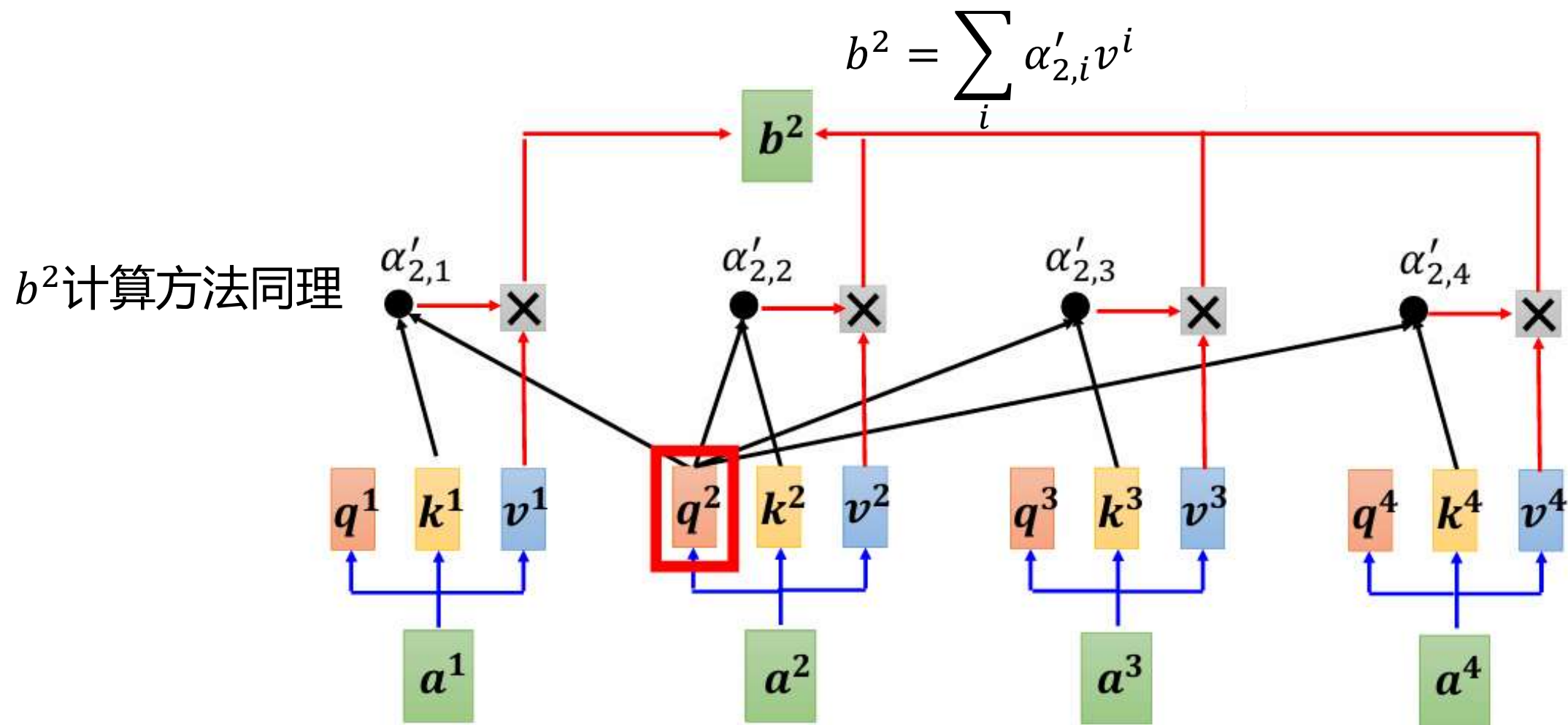
$$b^1 = \sum_i \alpha'_{1,i} v^i$$

相关性程度可以理解
为权重

比如 a^1 和 a^2 很相关
那么 b^2 的值就受 v^2 的
影响更大

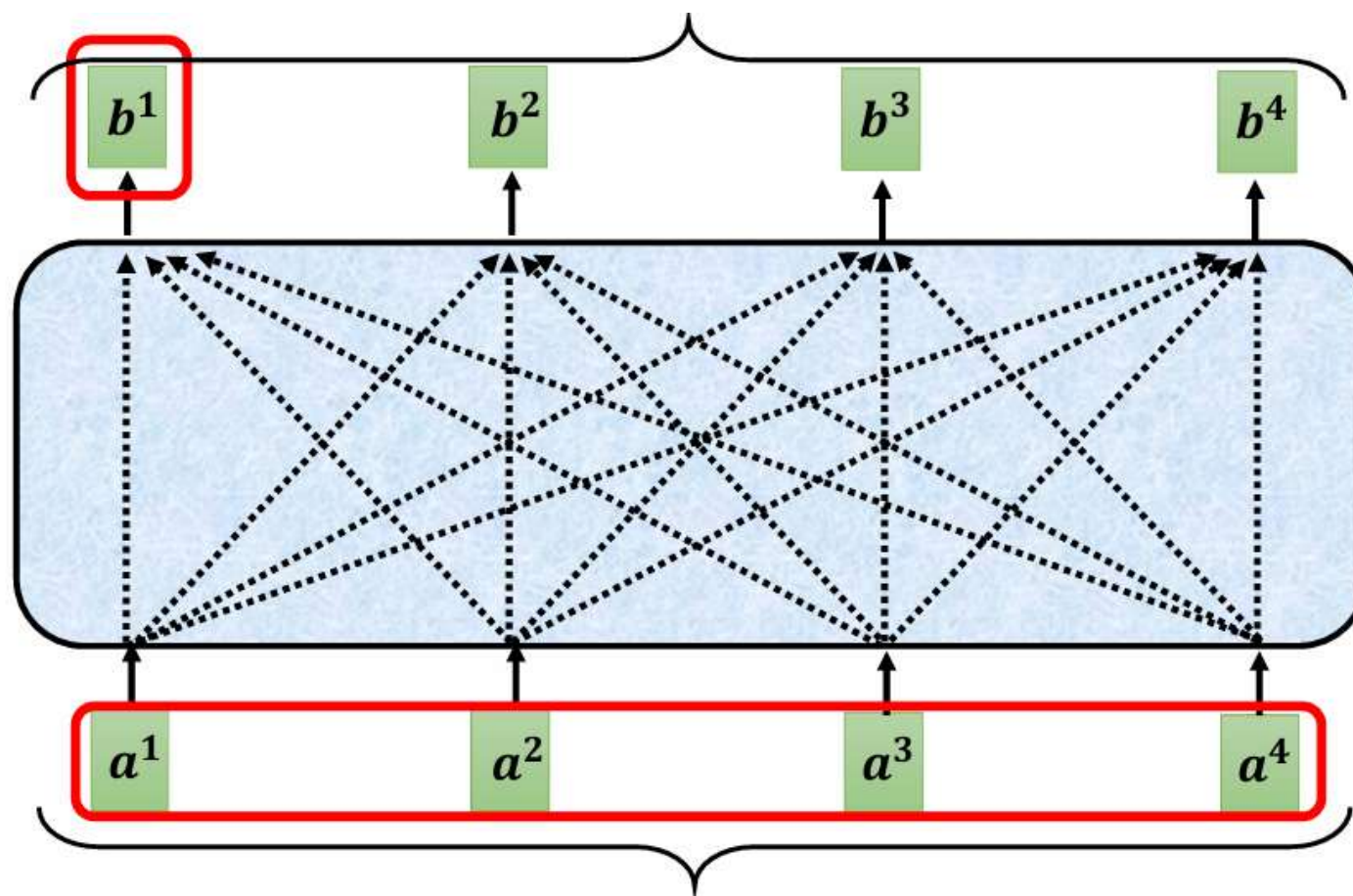
典型自回归网络结构-Attention

根据相关性程度提取关键信息



典型自回归网络结构-Attention

并行计算：同时对所有的输出进行计算



给定输入下，所有输出之间并没有依赖关系。

可以输入一个向量/一个隐藏层的输出

典型自回归网络结构-Attention

并行计算：同时对所有的输出进行计算

矩阵计算角度：

$$q^i = W^q a^i \quad \begin{matrix} q^1 & q^2 & q^3 & q^4 \end{matrix} = \begin{matrix} W^q & \begin{matrix} a^1 & a^2 & a^3 & a^4 \end{matrix} \end{matrix}$$

Q矩阵:Q 矩阵:I

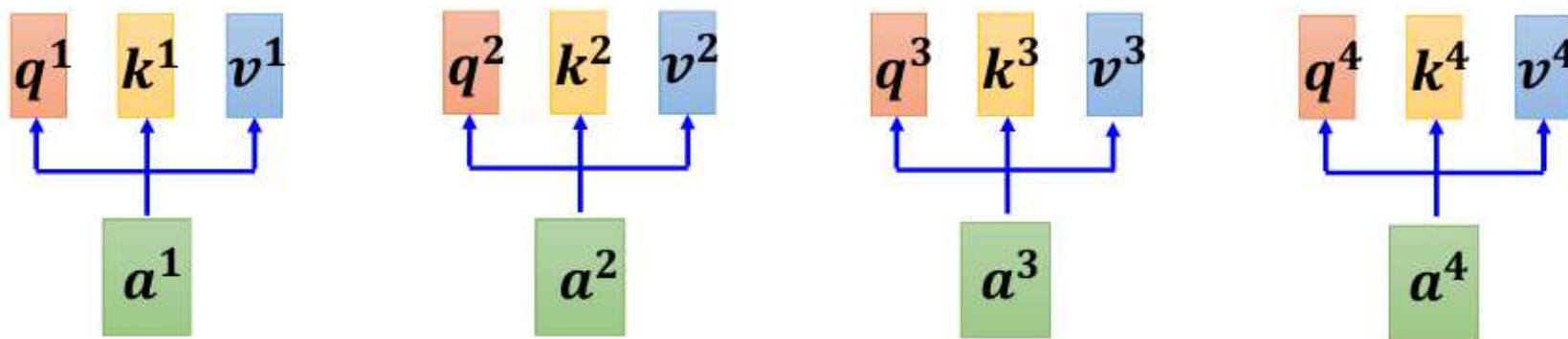
$$k^i = W^k a^i \quad \begin{matrix} k^1 & k^2 & k^3 & k^4 \end{matrix} = \begin{matrix} W^k & \begin{matrix} a^1 & a^2 & a^3 & a^4 \end{matrix} \end{matrix}$$

K矩阵:K I

Q、K、V三矩阵计算操作一致

$$v^i = W^v a^i \quad \begin{matrix} v^1 & v^2 & v^3 & v^4 \end{matrix} = \begin{matrix} W^v & \begin{matrix} a^1 & a^2 & a^3 & a^4 \end{matrix} \end{matrix}$$

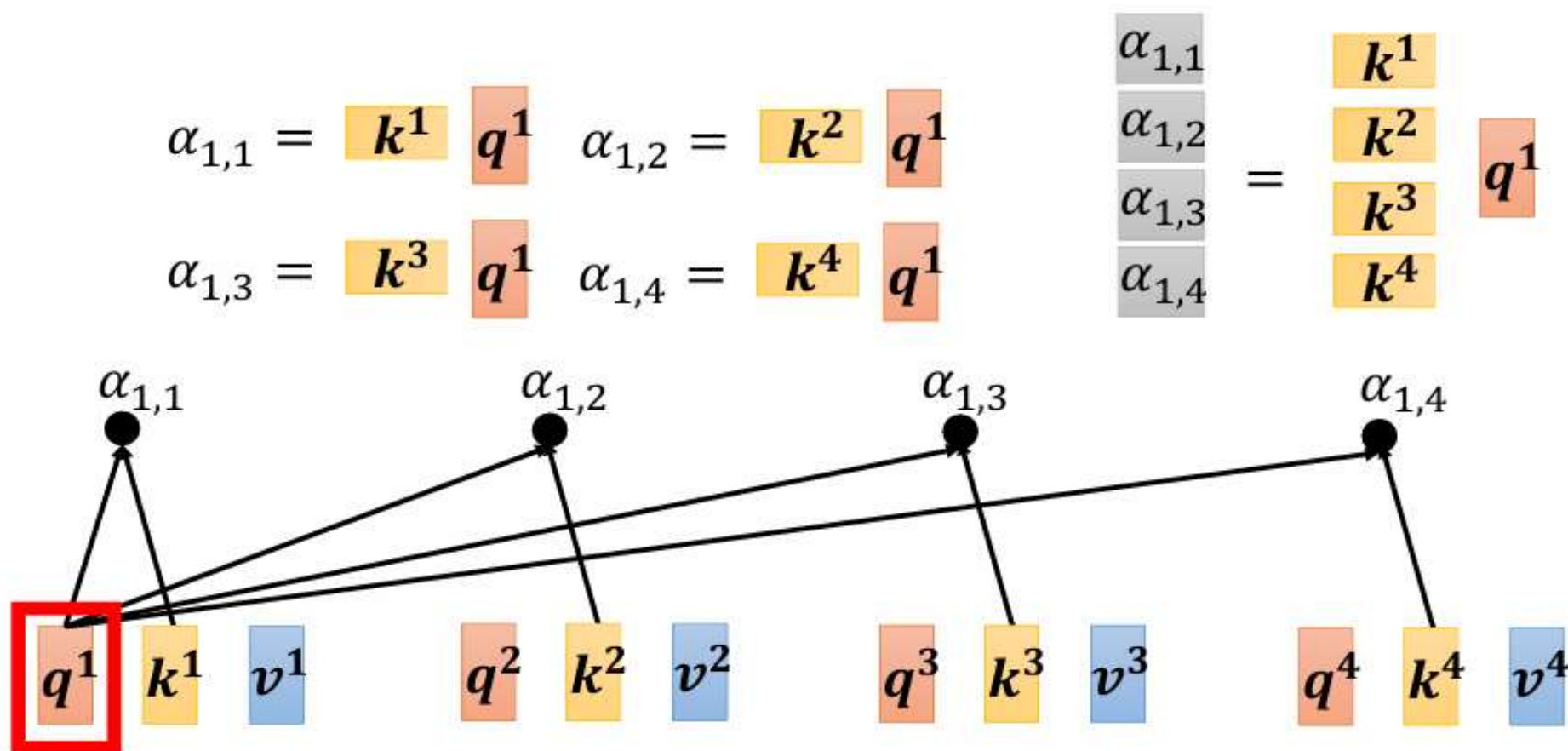
V矩阵:V I



典型自回归网络结构-Attention

并行计算：同时对所有的输出进行计算

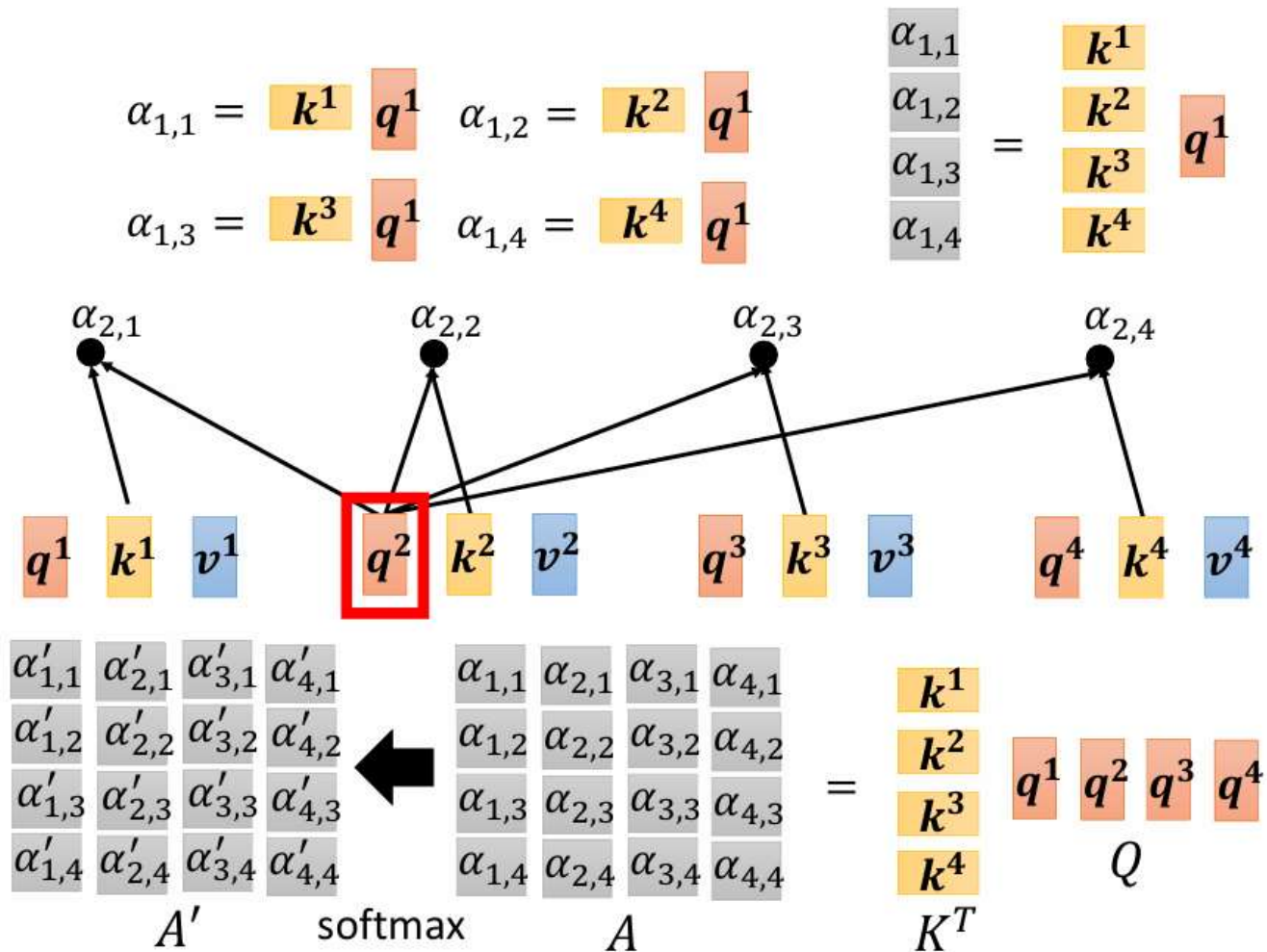
通过矩阵计算，求每两个向量之间的相关性程度（attention score）



典型自回归网络结构-Attention

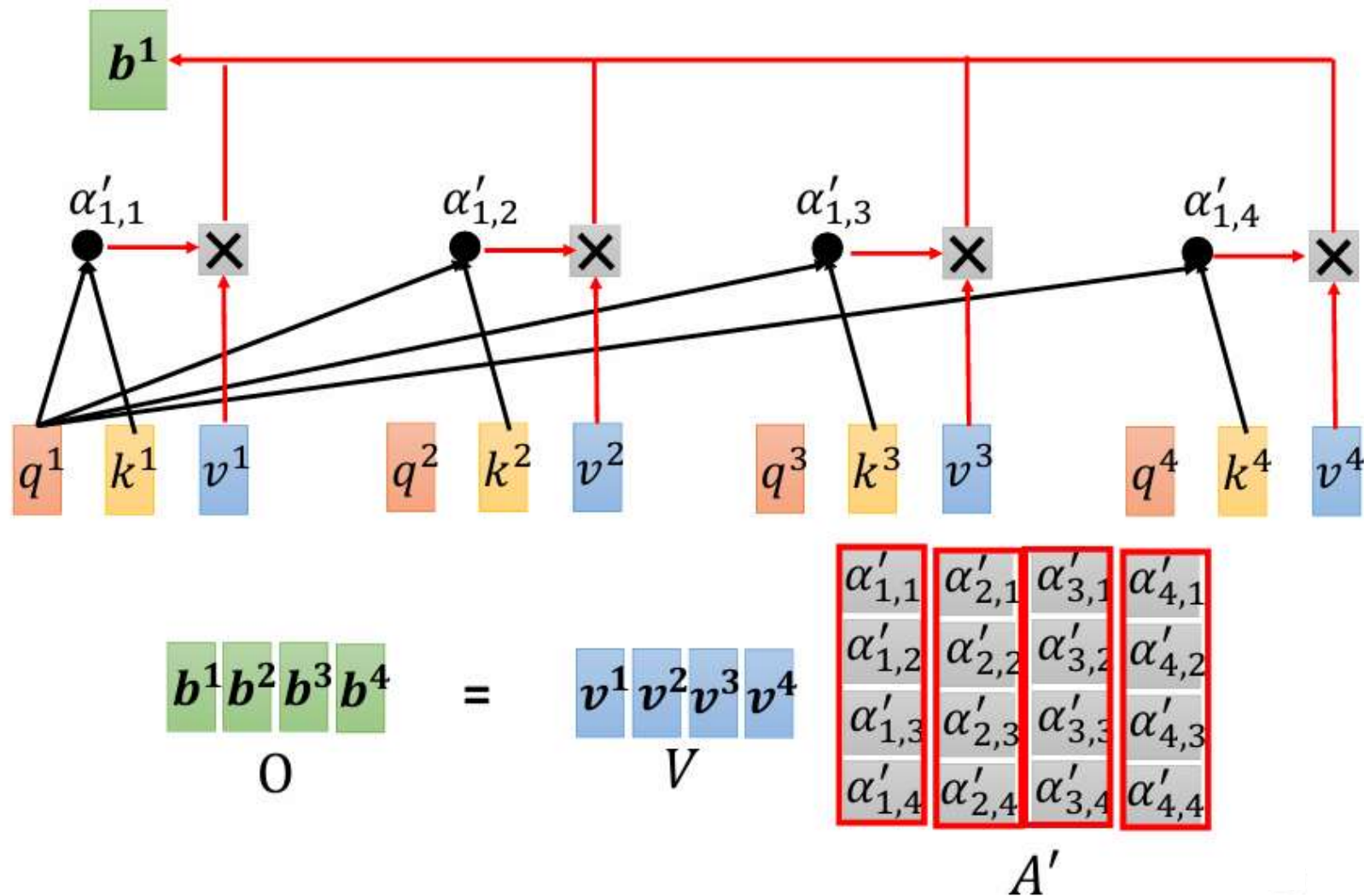
并行计算：同时对所有的输出进行计算

通过矩阵计算，求每两个向量之间的相关性程度（attention score）



典型自回归网络结构-Attention

并行计算：同时对所有的输出进行计算



典型自回归网络结构-Attention

自注意力机制的完整表述

步骤一：计算向量之间的相关性程度：

$$\begin{matrix} Q \\ K \\ V \end{matrix} = \begin{matrix} W^q \\ W^k \\ W^v \end{matrix} \begin{matrix} I \\ I \\ I \end{matrix} \quad \text{输入}$$

需要被训练的参数

$$A' \leftarrow A = K^T Q$$

Attention Matrix

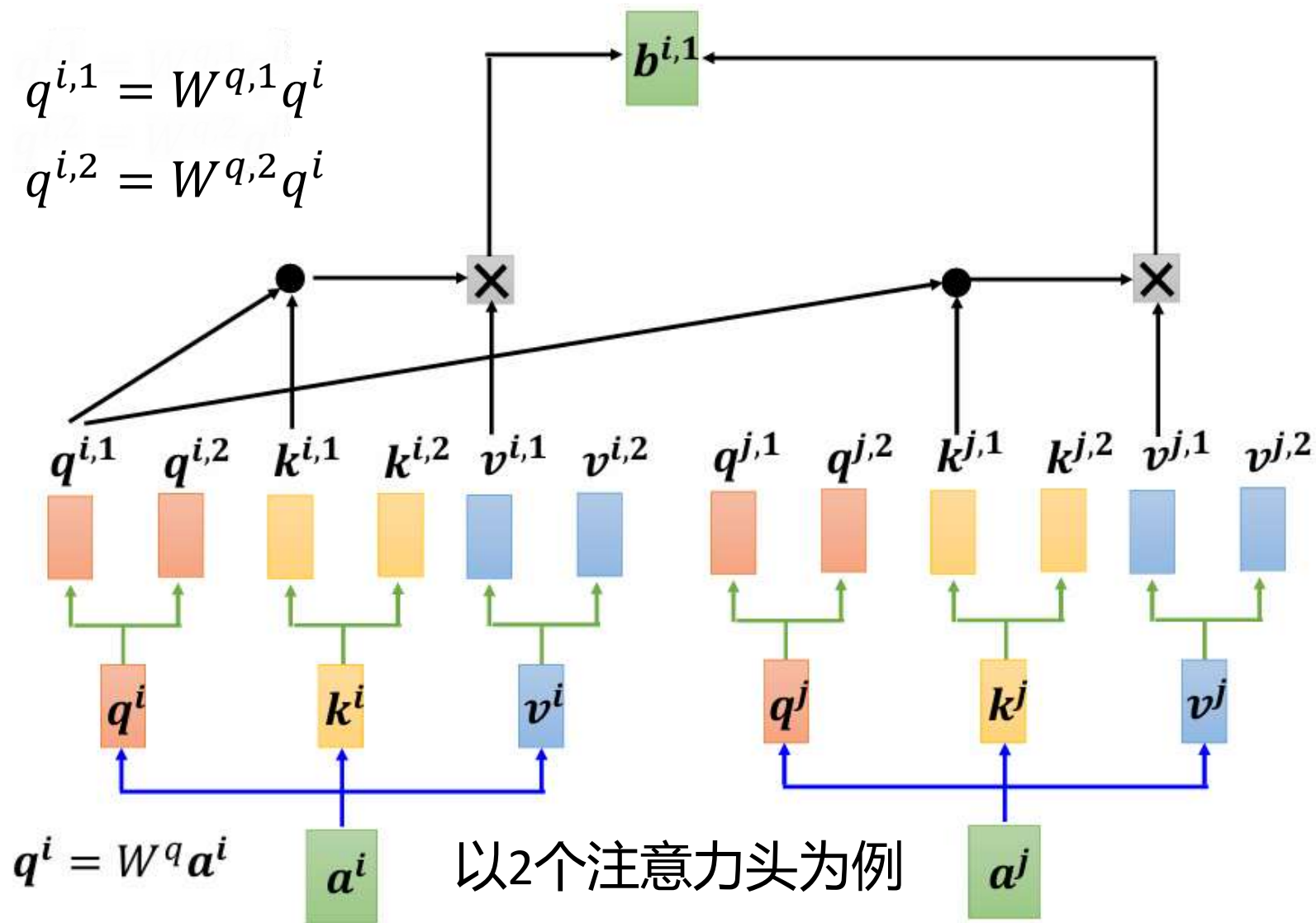
步骤二：根据相关性程度提取关键信息：

$$O = V A'$$

输出

典型自回归网络结构-Attention

多头自注意力

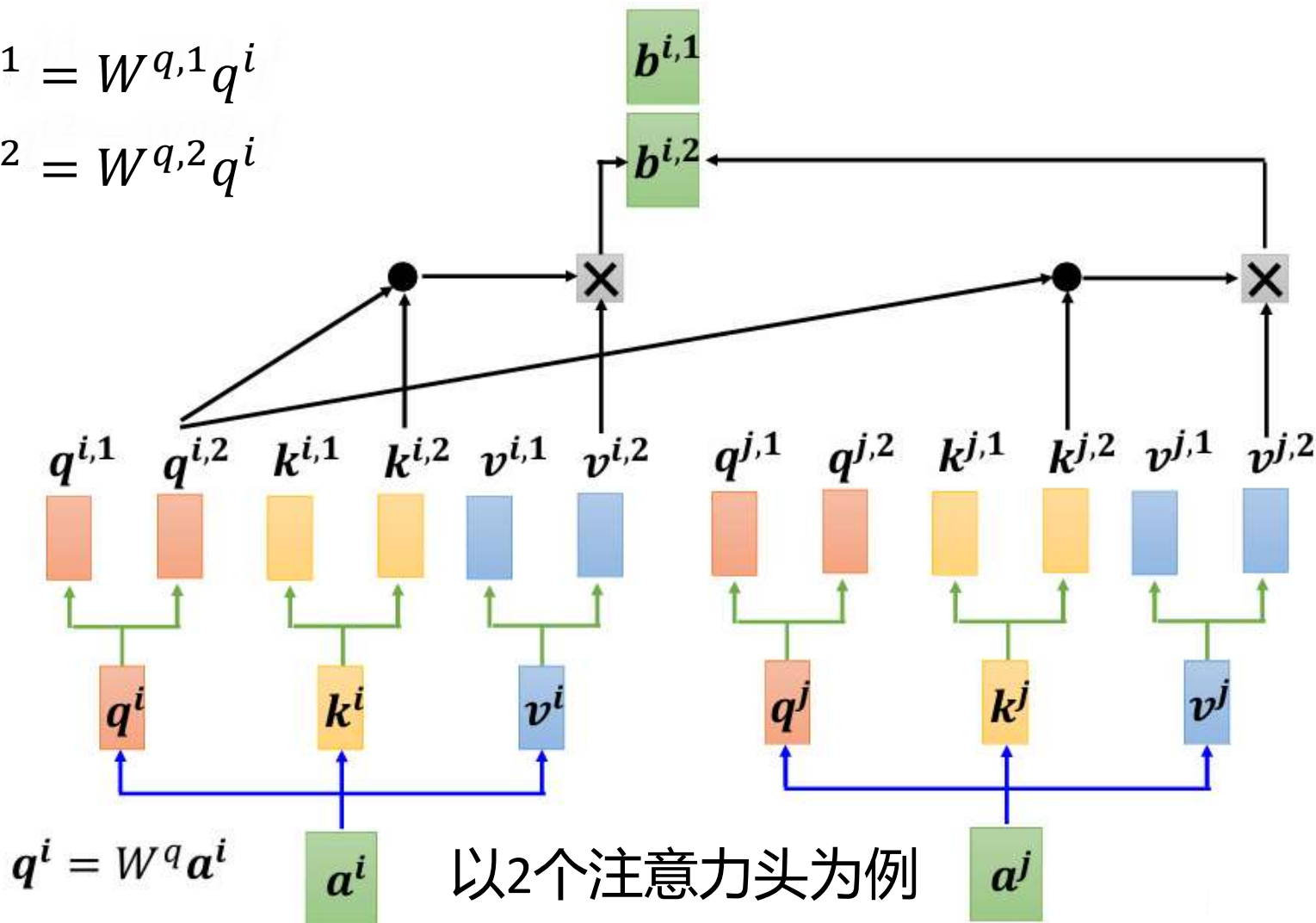


典型自回归网络结构-Attention

多头自注意力

$$q^{i,1} = W^{q,1} q^i$$

$$q^{i,2} = W^{q,2} q^i$$

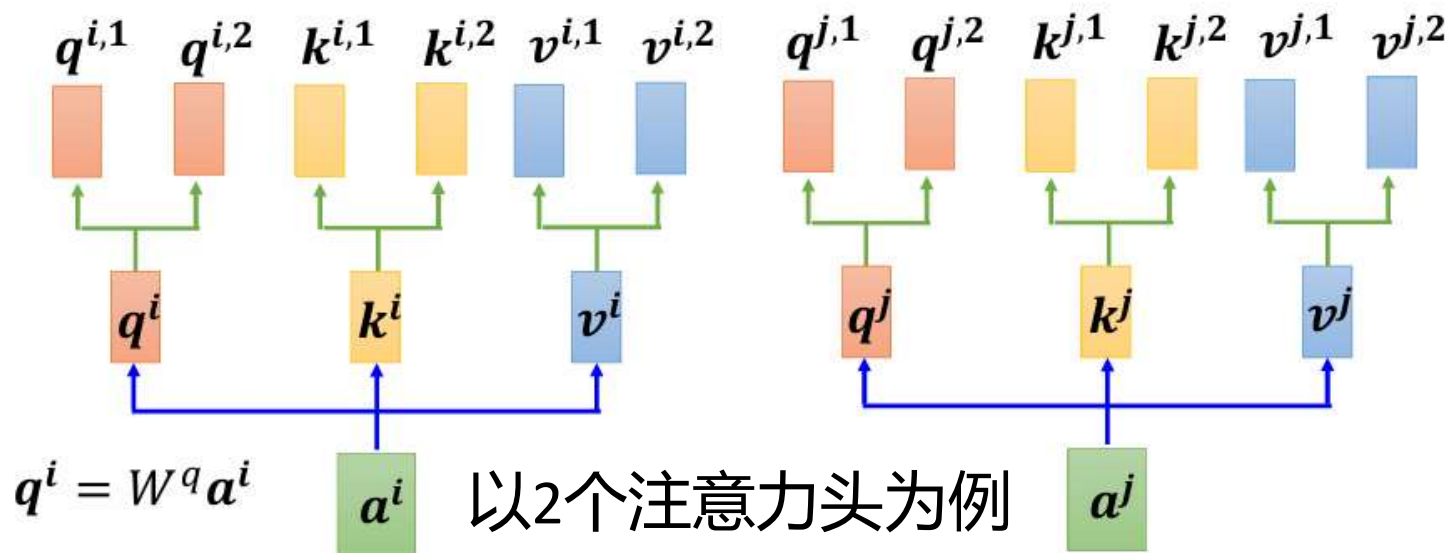


典型自回归网络结构-Attention

多头自注意力

$$b^i = W^O \begin{bmatrix} b^{i,1} \\ b^{i,2} \end{bmatrix}$$

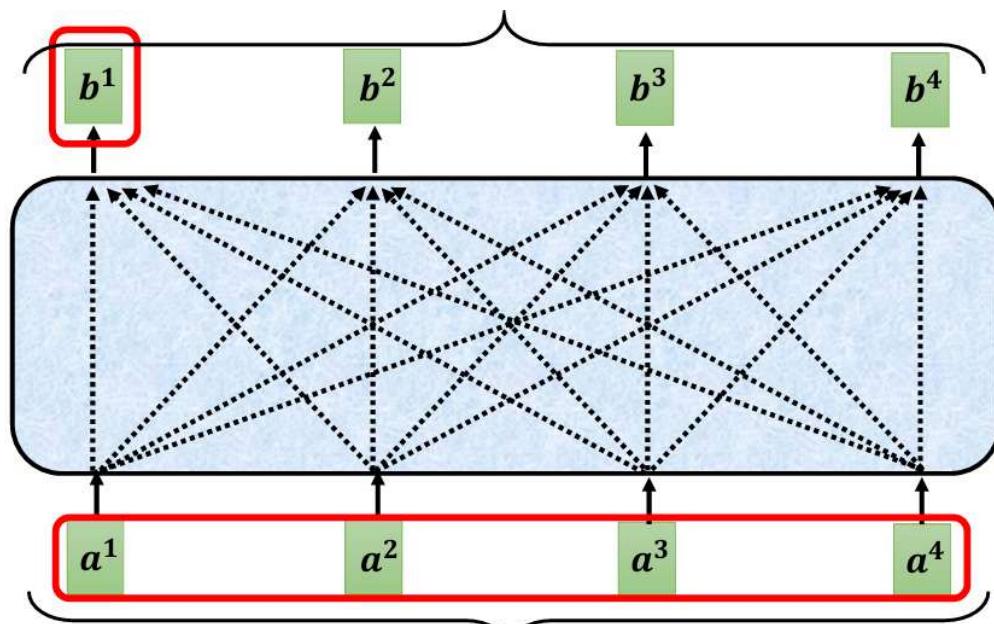
和单注意力头同理，可扩展至n个头



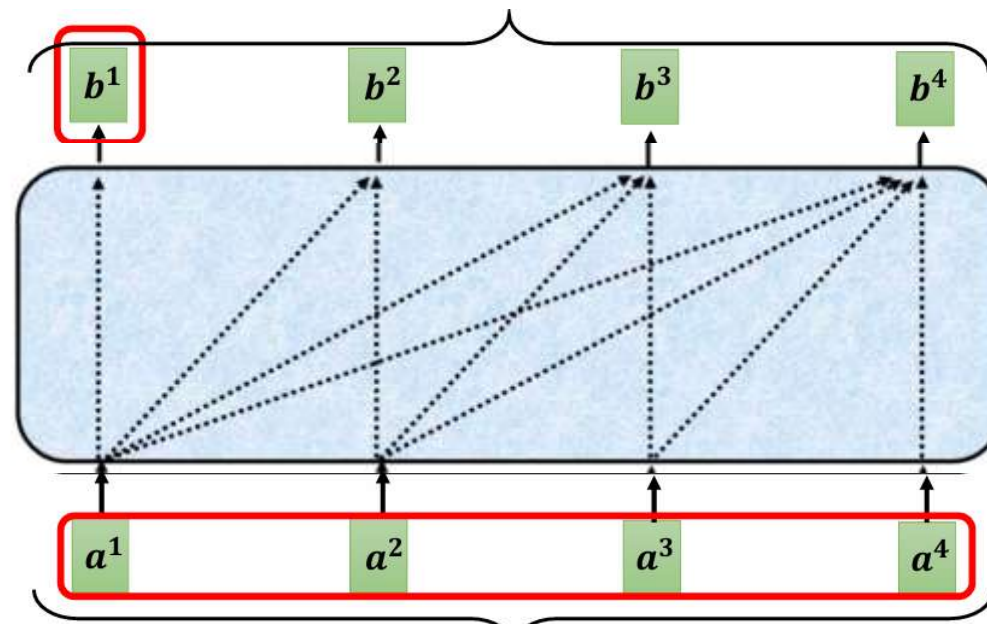
典型自回归网络结构-Attention

因果自注意力机制

当前词元只和其前面的词元关联



可以是输入层或隐藏层

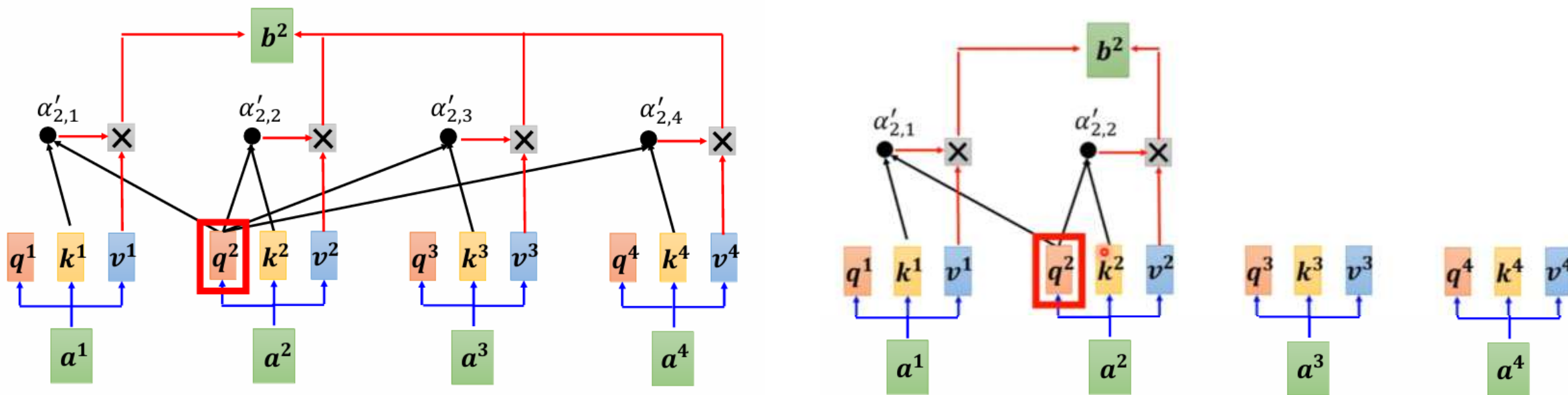


可以是输入层或隐藏层

典型自回归网络结构-Attention

因果自注意力机制

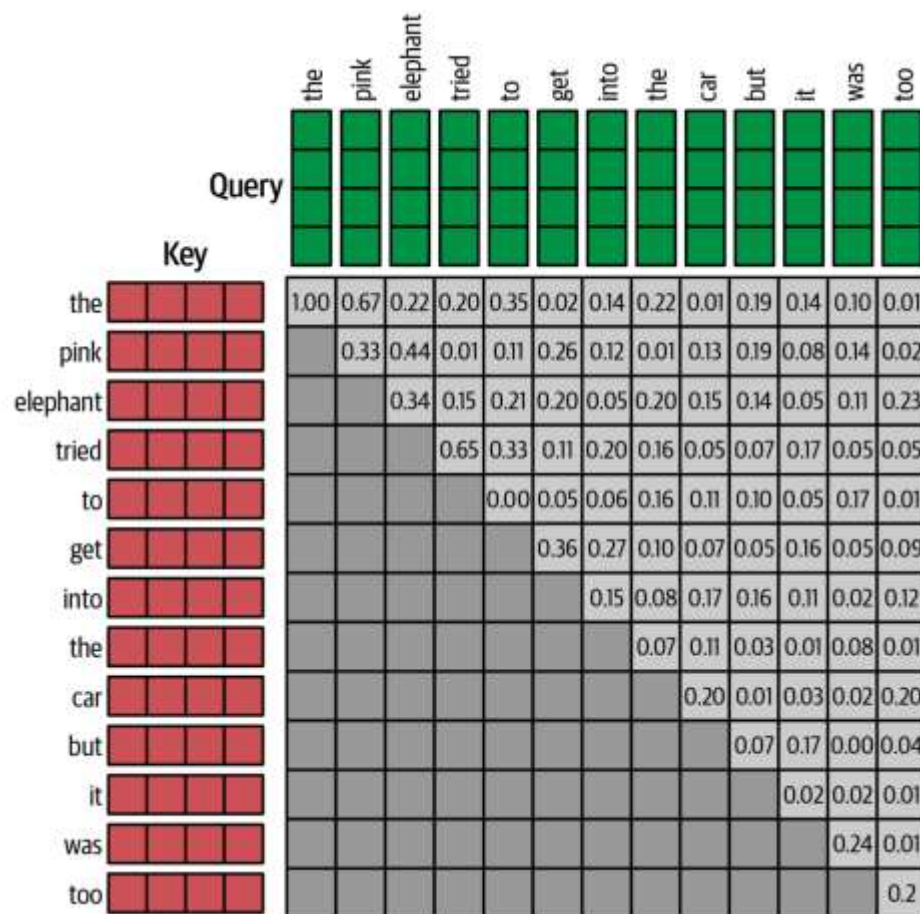
当前词元只和其前面的词元关联



典型自回归网络结构-Attention

因果自注意力机制

当前词元只和其前面的词元关联



限制关联矩阵为半三角形矩阵

$$A = K^T Q$$

需要被训练的参数

典型自回归网络结构-Transformer

Transformer 块

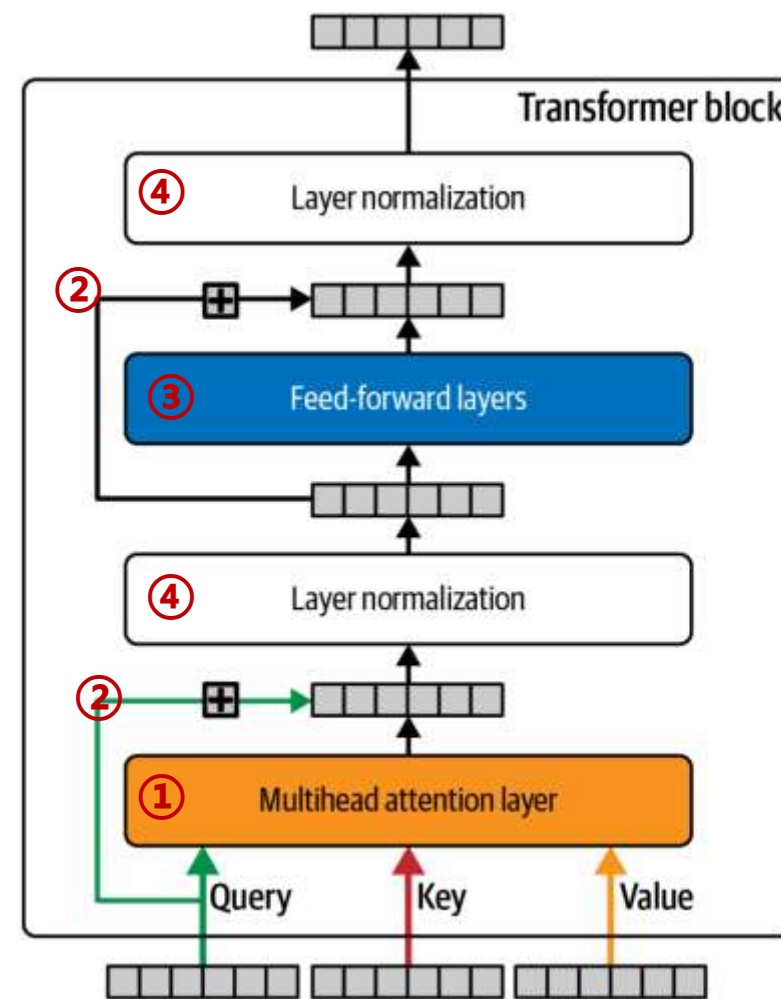
➤ Transformer 块是Transformer模型的基本组成单元

①多头注意力层：输入各词元信息交互

②跳跃链接：提高训练稳定性

③前馈全连接层：多层全连接网络

④标准化操作：对输入数值进行标准化处理

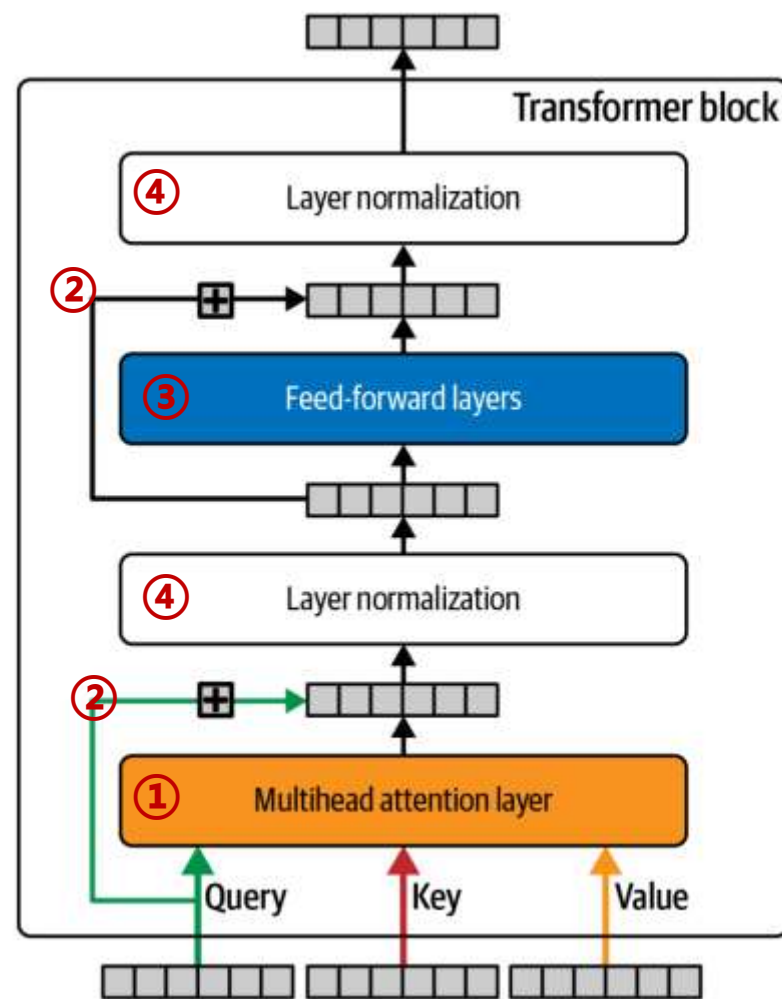
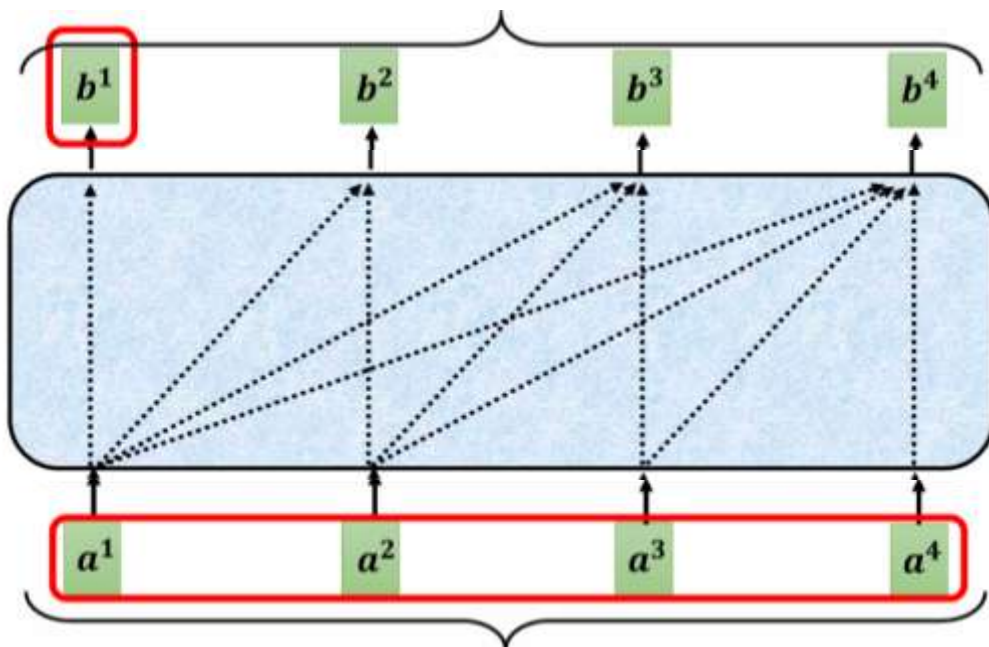


典型自回归网络结构-Transformer

Transformer 块

- Transformer 块是Transformer模型的基本组成单元

①多头注意力层：输入各词元信息交互



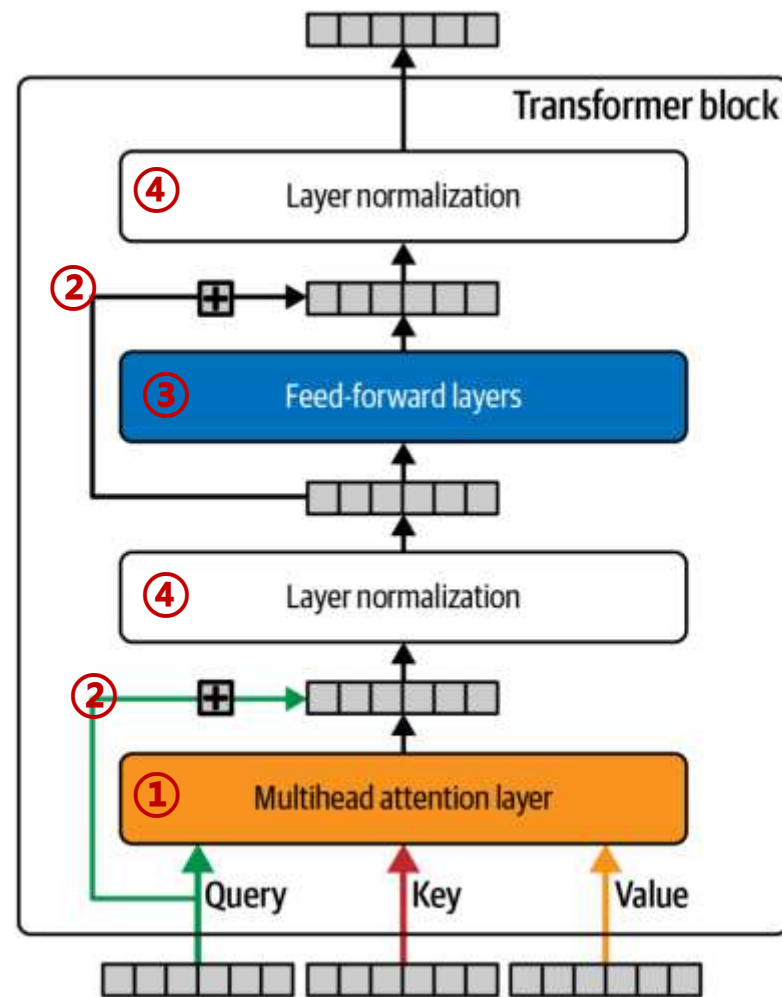
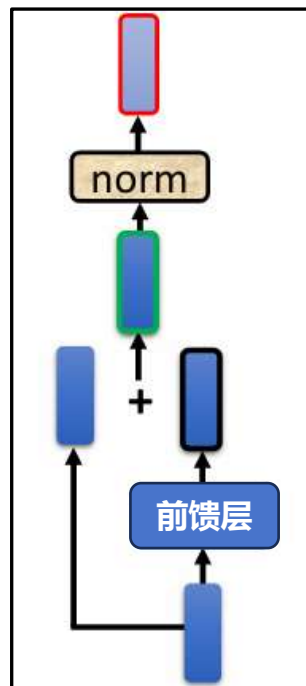
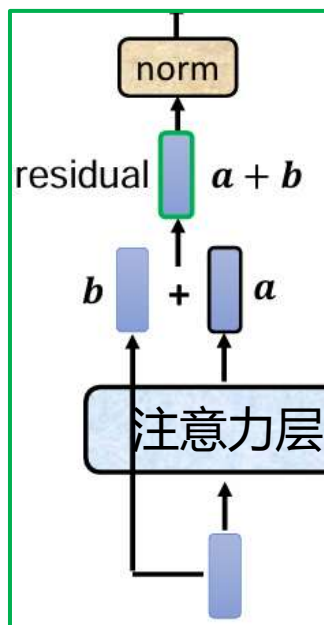
典型自回归网络结构-Transformer

Transformer 块

➤ Transformer 块是Transformer模型的基本组成单元

①多头注意力层：输入各词元信息交互

②跳跃链接：提高训练稳定性



典型自回归网络结构-Transformer

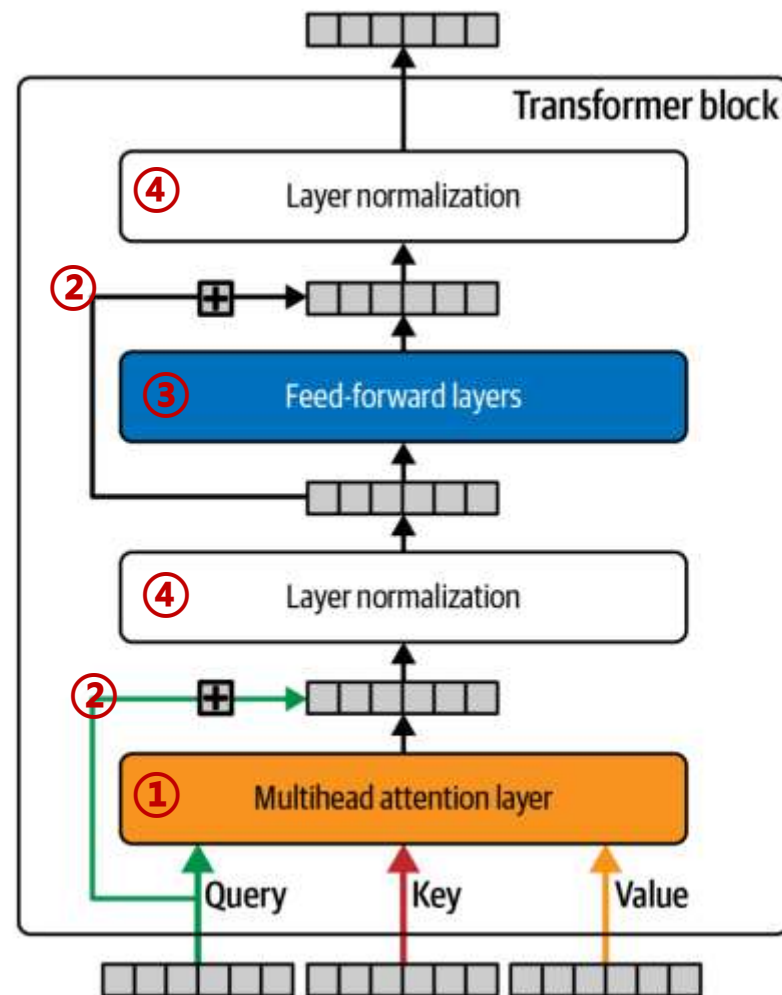
Transformer 块

➤ Transformer 块是Transformer模型的基本组成单元

①多头注意力层：输入各词元信息交互

②跳跃链接：提高训练稳定性

③前馈全连接层：多层全连接网络

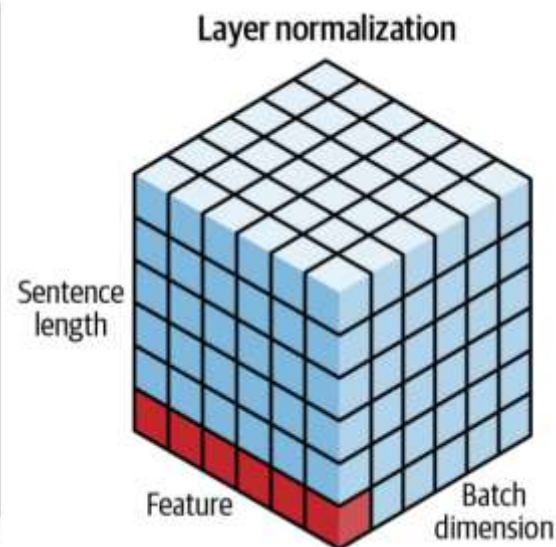
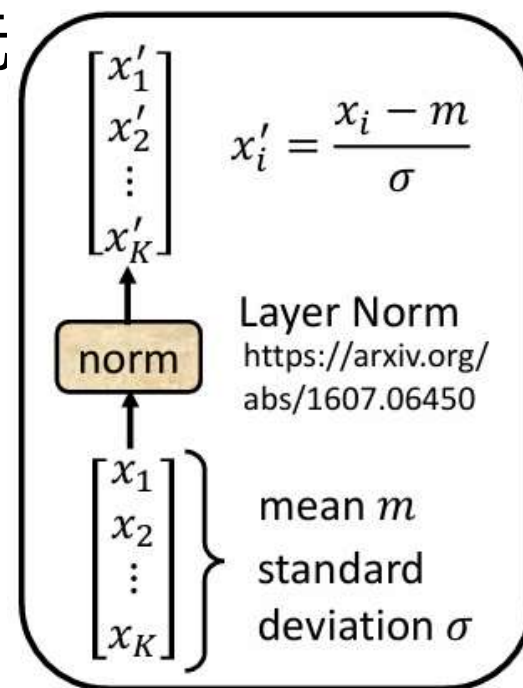


典型自回归网络结构-Transformer

Transformer 块

➤ Transformer 块是Transformer模型的基本组成单元

- ①多头注意力层：输入各词元信息交互
- ②跳跃链接：提高训练稳定性
- ③前馈全连接层：多层全连接网络
- ④标准化操作：对输入数值进行标准化处理

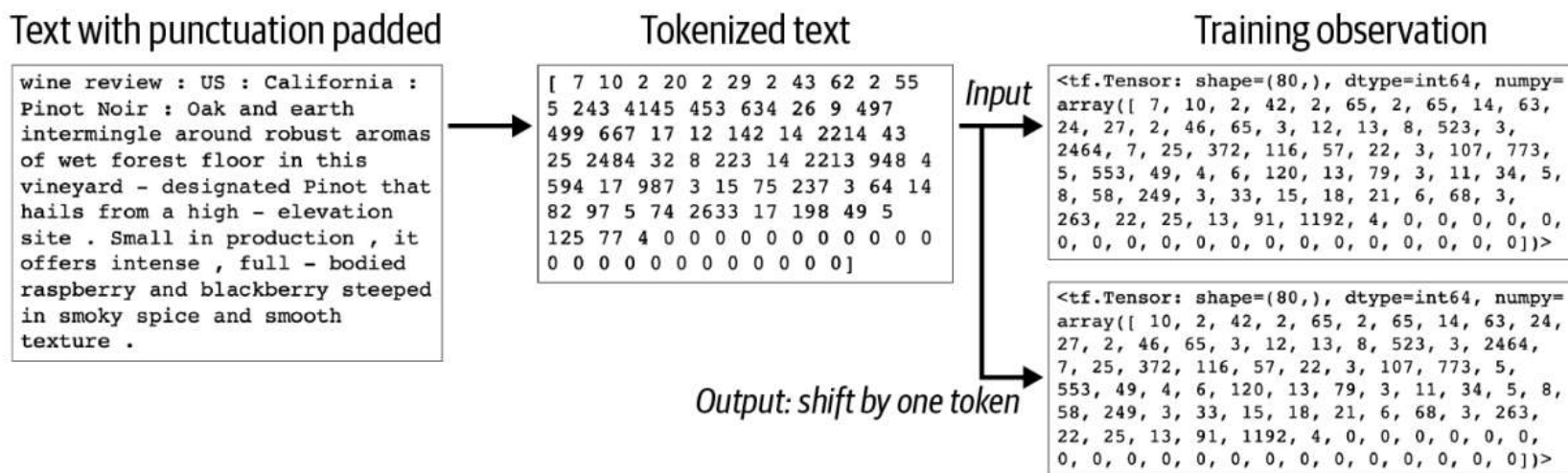


层标准化对每个词元特征向量进行归一化

典型自回归网络结构-mini-GPT

以葡萄酒评论数据集为例构建Transformer (mini-GPT)

数据集：葡萄酒评论数据集，包含130K葡萄酒评论文本数据



- **数据提取：**加载数据，提取葡萄酒的文本描述列表。
- **标点隔离：**在标点旁填充空格，使其被视为独立词元。
- **向量化：**进行分词 (Tokenize)，并统一序列长度 (Pad/Clip)。
- **构建训练集：**输入当前文本序列，输出向后位移一位的同源序列。

典型自回归网络结构-mini-GPT

Transformer block代码实现

1. 在初始化函数中定义网络层
2. 创建因果注意力掩码

```
array([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
       [0, 1, 1, 1, 1, 1, 1, 1, 1, 1],
       [0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
       [0, 0, 0, 1, 1, 1, 1, 1, 1, 1],
       [0, 0, 0, 0, 1, 1, 1, 1, 1, 1],
       [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 0, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]], dtype=int32)
```

因果掩码为一个半三角矩阵

```
class TransformerBlock(layers.Layer):
    def __init__(self, num_heads, key_dim, embed_dim, ff_dim, dropout_rate=0.1): ❶
        super(TransformerBlock, self).__init__()
        self.num_heads = num_heads
        self.key_dim = key_dim
        self.embed_dim = embed_dim
        self.ff_dim = ff_dim
        self.dropout_rate = dropout_rate
        self.attn = layers.MultiHeadAttention(
            num_heads, key_dim, output_shape = embed_dim
        )
        self.dropout_1 = layers.Dropout(self.dropout_rate)
        self.ln_1 = layers.LayerNormalization(epsilon=1e-6)
        self.ffn_1 = layers.Dense(self.ff_dim, activation="relu")
        self.ffn_2 = layers.Dense(self.embed_dim)
        self.dropout_2 = layers.Dropout(self.dropout_rate)
        self.ln_2 = layers.LayerNormalization(epsilon=1e-6)

    def call(self, inputs):
        input_shape = tf.shape(inputs)
        batch_size = input_shape[0]
        seq_len = input_shape[1]
        causal_mask = causal_attention_mask(
            batch_size, seq_len, seq_len, tf.bool
        ) ❷
        attention_output, attention_scores = self.attn(
            inputs,
            inputs,
            attention_mask=causal_mask,
            return_attention_scores=True
        ) ❸
        attention_output = self.dropout_1(attention_output)
        out1 = self.ln_1(inputs + attention_output) ❹
        ffn_1 = self.ffn_1(out1) ❺
        ffn_2 = self.ffn_2(ffn_1)
        ffn_output = self.dropout_2(ffn_2)
        return (self.ln_2(out1 + ffn_output), attention_scores) ❻
```


典型自回归网络结构-mini-GPT

Transformer block代码实现

1. 在初始化函数中定义网络层
2. 创建因果注意力掩码
3. 创建多头注意力机制，并指定注意力掩码
4. 第一个跳跃链接与标准化层
5. 前馈层
6. 第二个跳跃链接与标准化层

```
class TransformerBlock(layers.Layer):
    def __init__(self, num_heads, key_dim, embed_dim, ff_dim, dropout_rate=0.1): ❶
        super(TransformerBlock, self).__init__()
        self.num_heads = num_heads
        self.key_dim = key_dim
        self.embed_dim = embed_dim
        self.ff_dim = ff_dim
        self.dropout_rate = dropout_rate
        self.attn = layers.MultiHeadAttention(
            num_heads, key_dim, output_shape = embed_dim
        )
        self.dropout_1 = layers.Dropout(self.dropout_rate)
        self.ln_1 = layers.LayerNormalization(epsilon=1e-6)
        self.ffn_1 = layers.Dense(self.ff_dim, activation="relu")
        self.ffn_2 = layers.Dense(self.embed_dim)
        self.dropout_2 = layers.Dropout(self.dropout_rate)
        self.ln_2 = layers.LayerNormalization(epsilon=1e-6)

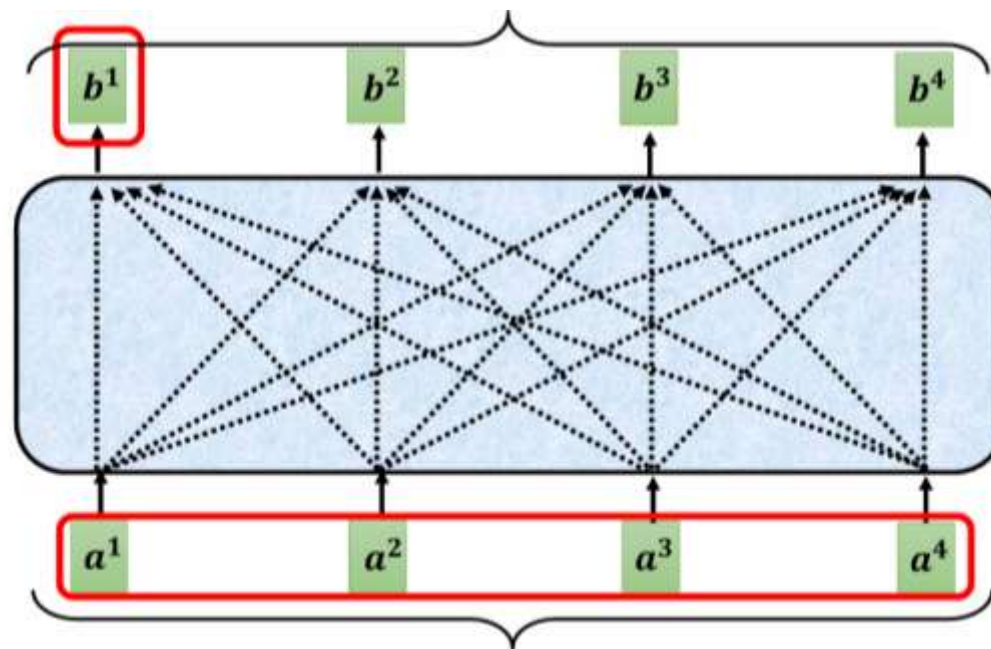
    def call(self, inputs):
        input_shape = tf.shape(inputs)
        batch_size = input_shape[0]
        seq_len = input_shape[1]
        causal_mask = causal_attention_mask(
            batch_size, seq_len, seq_len, tf.bool
        ) ❷
        attention_output, attention_scores = self.attn(
            inputs,
            inputs,
            attention_mask=causal_mask,
            return_attention_scores=True
        ) ❸
        attention_output = self.dropout_1(attention_output)
        out1 = self.ln_1(inputs + attention_output) ❹
        ffn_1 = self.ffn_1(out1) ❺
        ffn_2 = self.ffn_2(ffn_1)
        ffn_output = self.dropout_2(ffn_2)
        return (self.ln_2(out1 + ffn_output), attention_scores) ❻
```

典型自回归网络结构-mini-GPT

位置编码

为每个词元注入位置信息，区分单词在不同位置上的作用效果

- 自注意力机制的问题：
 - 并行计算所有词元 $\rightarrow$ 丢失原始输入顺序信息
 - 等价于乱序词集合



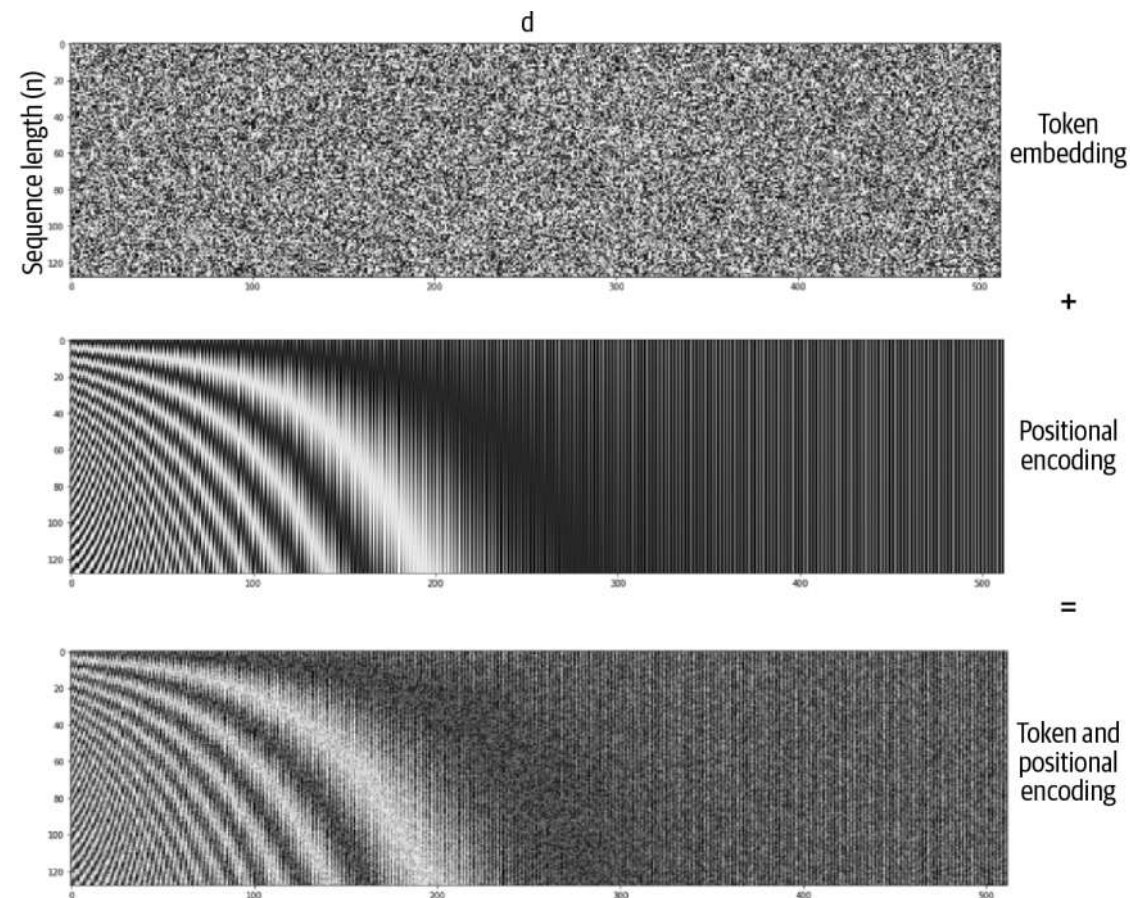
典型自回归网络结构-mini-GPT

位置编码

为每个词元注入位置信息，区分单词在不同位置上的作用效果

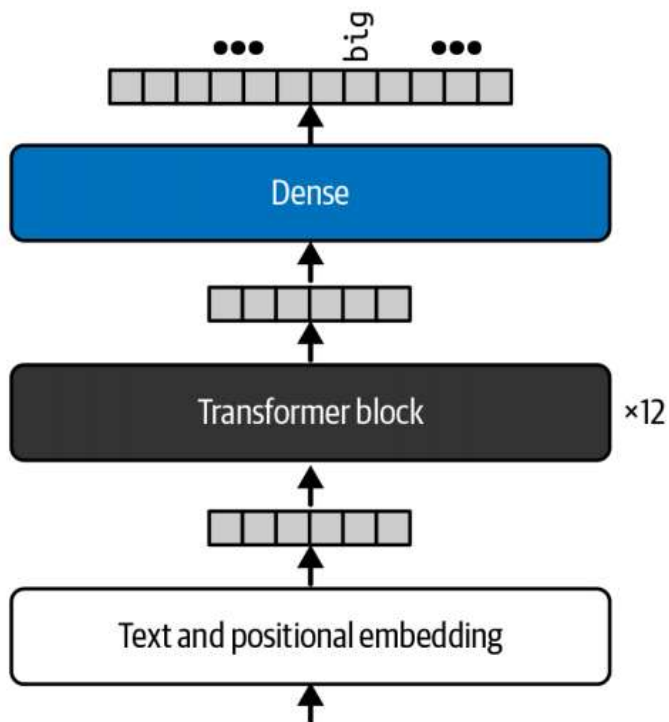
```
class TokenAndPositionEmbedding(layers.Layer):
    def __init__(self, maxlen, vocab_size, embed_dim):
        super(TokenAndPositionEmbedding, self).__init__()
        self.maxlen = maxlen
        self.vocab_size = vocab_size
        self.embed_dim = embed_dim
        self.token_emb = layers.Embedding(
            input_dim=vocab_size, output_dim=embed_dim
        ) ①
        self.pos_emb = layers.Embedding(input_dim=maxlen, output_dim=embed_dim) ②

    def call(self, x):
        maxlen = tf.shape(x)[-1]
        positions = tf.range(start=0, limit=maxlen, delta=1)
        positions = self.pos_emb(positions)
        x = self.token_emb(x)
        return x + positions ③
```



典型自回归网络结构-mini-GPT

Mini-GPT训练



the pink elephant tried to get into the car but it was too

实践中只用1层transformer

```
MAX_LEN = 80
VOCAB_SIZE = 10000
EMBEDDING_DIM = 256
N_HEADS = 2
KEY_DIM = 256
FEED_FORWARD_DIM = 256
```

```
inputs = layers.Input(shape=(None,), dtype=tf.int32) ①
x = TokenAndPositionEmbedding(MAX_LEN, VOCAB_SIZE, EMBEDDING_DIM)(inputs) ②
x, attention_scores = TransformerBlock(
    N_HEADS, KEY_DIM, EMBEDDING_DIM, FEED_FORWARD_DIM
)(x) ③
outputs = layers.Dense(VOCAB_SIZE, activation = 'softmax')(x) ④
gpt = models.Model(inputs=inputs, outputs=[outputs, attention]) ⑤
gpt.compile("adam", loss=[losses.SparseCategoricalCrossentropy(), None]) ⑥
gpt.fit(train_ds, epochs=5)
```

Mini-GPT生成效果

➤ LSTM相同流程：

- **预测下一个词：** 将现有的单词序列输入网络，让模型预测紧接其后的下一个单词。
- **循环迭代：** 将新预测出的单词追加到序列末尾，作为新的输入重复上述过程以生成后续内容。

Generated text:

wine review : us : washington : chenin blanc : a light , medium - bodied wine , this light - bodied expression is not a lot of enjoyment . it ' s simple with butter and vanilla flavors that frame mixed with expressive fruit . it ' s juicy and tangy with a lemon lingers on the finish .

Generated text:

wine review : italy : piedmont : nebbiolo : this opens with aromas of french oak , menthol and a whiff of toast . the straightforward palate offers red cherry , black raspberry jam and a hint of star anise alongside firm but rather fleeting tannins , drink through 2016 .

文体上都与原始训练集中的葡萄酒评论相似

作业： GPT的代码实现

总结

➤ 联合分布建模

- 通过条件概率乘积（链式法则）建模联合分布

$$p(x_1, x_2, \dots, x_n) = \prod_{i=1}^n p(x_i | x_1, x_2, \dots, x_{i-1})$$

➤ 自回归模型核心思想

- 利用历史信息预测当前元素
- 推理时自回归，训练时常用Teacher Forcing
- 可适用于序列、图像、隐变量等多种数据形

➤ 典型网络结构

- RNN：具备记忆能力，适合处理序列
- CNN / Masked CNN：局部建模，可并行训练
- Transformer（解码器）：
 - 使用Masked Self-Attention防止未来信息泄露
 - 支持长距离依赖建模，适合大规模生成任务